

CICIDS2017_Friday_DDoS_EDA

April 20, 2026

```
[1]: import os
from pathlib import Path
import warnings

os.environ.setdefault("MPLCONFIGDIR", "/tmp")
warnings.filterwarnings("ignore", category=FutureWarning)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, Markdown

pd.set_option("display.max_columns", 120)
pd.set_option("display.max_rows", 80)
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")
sns.set_theme(style="whitegrid", context="notebook")

# Print-friendly plotting helpers for black-and-white Xerox copies.
PRINT_HATCHES = ["", "///", "\\\\", "xx", "..", "++", "--", "oo", "**", "00"]
PRINT_MARKERS = ["o", "s", "^", "D", "P", "X", "v", "<", ">", "*"]
PRINT_GRAYS = ["#111111", "#444444", "#777777", "#AAAAAA", "#DDDDDD"]

plt.rcParams.update({
    "figure.facecolor": "white",
    "axes.facecolor": "white",
    "axes.edgecolor": "#111111",
    "axes.labelcolor": "#111111",
    "xtick.color": "#111111",
    "ytick.color": "#111111",
    "grid.color": "#D0D0D0",
})

def print_palette(n):
    if n <= 0:
        return []
    return [PRINT_GRAYS[i % len(PRINT_GRAYS)] for i in range(n)]
```

```

def apply_bar_hatches(ax, hatches=PRINT_HATCHES):
    for i, patch in enumerate(ax.patches):
        patch.set_facecolor("white")
        patch.set_edgecolor("#111111")
        patch.set_linewidth(1.0)
        patch.set_hatch(hatches[i % len(hatches)])
    return ax

def boxplot_print(data, x, y, ax=None, showfliers=False):
    ax = ax or plt.gca()
    sns.boxplot(
        data=data,
        x=x,
        y=y,
        ax=ax,
        color="white",
        showfliers=showfliers,
        boxprops={"facecolor": "white", "edgecolor": "#111111", "linewidth": 1.
↪0},
        medianprops={"color": "#111111", "linewidth": 1.3},
        whiskerprops={"color": "#111111", "linewidth": 1.0},
        capprops={"color": "#111111", "linewidth": 1.0},
    )
    for i, patch in enumerate(ax.patches):
        patch.set_hatch(PRINT_HATCHES[i % len(PRINT_HATCHES)])
        patch.set_edgecolor("#111111")
    return ax

def scatter_with_markers(data, x, y, hue, title, xlabel=None, ylabel=None, ↵
↪max_legend_items=12):
    fig, ax = plt.subplots(figsize=(8.5, 6))
    labels = data[hue].astype("string").fillna("<NA>")
    order = labels.value_counts().index.tolist()
    if len(order) > max_legend_items:
        keep = set(order[:max_legend_items - 1])
        plot_data = data.copy()
        plot_data[hue] = labels.where(labels.isin(keep), "Other")
        labels = plot_data[hue].astype("string")
        order = labels.value_counts().index.tolist()
    else:
        plot_data = data.copy()
        plot_data[hue] = labels

    for i, label in enumerate(order):
        subset = plot_data[plot_data[hue].astype("string") == str(label)]
        ax.scatter(

```

```

        subset[x],
        subset[y],
        marker=PRINT_MARKERS[i % len(PRINT_MARKERS)],
        facecolors="none",
        edgecolors="#111111",
        linewidths=0.75,
        alpha=0.7,
        s=24,
        label=str(label),
    )
ax.set_title(title)
ax.set_xlabel(xlabel or x)
ax.set_ylabel(ylabel or y)
ax.legend(title=hue, bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
return ax

RANDOM_STATE = 42
PLOT_SAMPLE_SIZE = 50_000

def find_data_file(filename):
    candidates = [
        Path(filename),
        Path("eda") / filename,
        Path("..") / "eda" / filename,
    ]
    for candidate in candidates:
        if candidate.exists():
            return candidate
    raise FileNotFoundError(f"Could not find {filename}. Tried: {candidates}")

def display_basic_profile(df, target_col=None):
    memory_mb = df.memory_usage(deep=True).sum() / (1024 ** 2)
    profile = pd.DataFrame({
        "metric": ["rows", "columns", "memory_mb", "duplicate_rows",
↪ "total_missing_cells"],
        "value": [len(df), df.shape[1], round(memory_mb, 2), int(df.
↪ duplicated().sum()), int(df.isna().sum().sum())],
    })
    display(profile)
    if target_col and target_col in df.columns:
        target_summary = (
            df[target_col]
            .value_counts(dropna=False)
            .rename_axis(target_col)

```

```

        .reset_index(name="count")
    )
    target_summary["percent"] = (target_summary["count"] / len(df) * 100).
↳round(2)
    display(target_summary)

def get_numeric_columns(df, exclude=None):
    exclude = set(exclude or [])
    return [c for c in df.select_dtypes(include=np.number).columns if c not in
↳exclude]

def get_categorical_columns(df, exclude=None):
    exclude = set(exclude or [])
    return [c for c in df.select_dtypes(include=["object", "string",
↳"category", "bool"]).columns if c not in exclude]

def sampled(df, n=PLOT_SAMPLE_SIZE):
    if len(df) <= n:
        return df.copy()
    return df.sample(n=n, random_state=RANDOM_STATE)

def plot_target_distribution(df, target_col, title):
    order = df[target_col].value_counts(dropna=False).index
    plt.figure(figsize=(8, 4.5))
    ax = sns.countplot(data=df, x=target_col, order=order, color="white",
↳edgecolor="#111111")
    apply_bar_hatches(ax)
    ax.set_title(title)
    ax.set_xlabel(target_col)
    ax.set_ylabel("Rows")
    ax.tick_params(axis="x", rotation=30)
    for container in ax.containers:
        ax.bar_label(container, fmt="%d", fontsize=9)
    plt.tight_layout()
    plt.show()

def plot_missing_values(df):
    missing = df.isna().sum().sort_values(ascending=False)
    missing = missing[missing > 0].head(20)
    if missing.empty:
        display(Markdown("No missing values after loading and standard cleanup.
↳"))
    return
    plt.figure(figsize=(10, max(3, len(missing) * 0.3)))
    ax = sns.barplot(x=missing.values, y=missing.index, color="white",
↳edgecolor="#111111")

```

```

    apply_bar_hatches(ax)
    plt.title("Top columns by missing value count")
    plt.xlabel("Missing cells")
    plt.ylabel("")
    plt.tight_layout()
    plt.show()

def plot_numeric_distributions(df, columns, title):
    if not columns:
        display(Markdown("No numeric columns available for distribution plots.
↪"))
        return
    sample = sampled(df)
    n_cols = min(3, len(columns))
    n_rows = int(np.ceil(len(columns) / n_cols))
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(5 * n_cols, 3.5 * n_rows))
    axes = np.array(axes).reshape(-1)
    for ax, col in zip(axes, columns):
        values = pd.to_numeric(sample[col], errors="coerce").replace([np.inf,
↪-np.inf], np.nan).dropna()
        if values.empty:
            ax.set_visible(False)
            continue
        label = col
        if values.min() >= 0 and values.quantile(0.99) > max(values.median() *
↪10, 10):
            values = np.log1p(values)
            label = f"log1p({col})"
        sns.histplot(values, bins=40, kde=True, ax=ax, color="#D9D9D9",
↪edgecolor="#111111", line_kws={"color": "#111111"})
        ax.set_title(col)
        ax.set_xlabel(label)
    for ax in axes[len(columns):]:
        ax.set_visible(False)
    fig.suptitle(title, y=1.02)
    plt.tight_layout()
    plt.show()

def plot_categorical_top_counts(df, columns, title):
    if not columns:
        display(Markdown("No categorical columns available for count plots."))
        return
    n_cols = min(2, len(columns))
    n_rows = int(np.ceil(len(columns) / n_cols))
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(6.5 * n_cols, 4 * n_rows))
    axes = np.array(axes).reshape(-1)
    for ax, col in zip(axes, columns):

```

```

        counts = df[col].astype("string").fillna("<NA>").value_counts().head(12)
        sns.barplot(x=counts.values, y=counts.index, ax=ax, color="white",
↪edgecolor="#111111")
        apply_bar_hatches(ax)
        ax.set_title(col)
        ax.set_xlabel("Rows")
        ax.set_ylabel("")
    for ax in axes[len(columns):]:
        ax.set_visible(False)
    fig.suptitle(title, y=1.02)
    plt.tight_layout()
    plt.show()

def target_association(df, target_col, numeric_cols, exclude=None):
    exclude = set(exclude or []) | {target_col}
    numeric_cols = [c for c in numeric_cols if c not in exclude]
    if target_col not in df.columns or not numeric_cols:
        return pd.Series(dtype="float64")
    y = pd.Series(pd.factorize(df[target_col].astype("string"))[0], index=df.
↪index)
    values = df[numeric_cols].replace([np.inf, -np.inf], np.nan)
    corr = values.corrwith(y).abs().sort_values(ascending=False)
    return corr.dropna()

def plot_correlation_heatmap(df, numeric_cols, title, max_features=18):
    numeric_cols = list(numeric_cols)[:max_features]
    if len(numeric_cols) < 2:
        display(Markdown("Not enough numeric columns for a correlation heatmap.
↪"))
    return
    corr = df[numeric_cols].replace([np.inf, -np.inf], np.nan).corr()
    plt.figure(figsize=(12, 9))
    sns.heatmap(corr, cmap="Greys", center=0, linewidths=0.3, square=False,
↪linecolor="#CCCCCC")
    plt.title(title)
    plt.tight_layout()
    plt.show()

def plot_numeric_by_target(df, target_col, columns, title):
    if not columns:
        display(Markdown("No numeric columns selected for target comparison."))
        return
    sample = sampled(df, 30_000)
    n_cols = min(3, len(columns))
    n_rows = int(np.ceil(len(columns) / n_cols))
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(5 * n_cols, 3.8 * n_rows))
    axes = np.array(axes).reshape(-1)

```

```

for ax, col in zip(axes, columns):
    values = sample[[target_col, col]].copy()
    values[col] = pd.to_numeric(values[col], errors="coerce").replace([np.
↳inf, -np.inf], np.nan)
    values = values.dropna(subset=[col])
    if values.empty:
        ax.set_visible(False)
        continue
    if values[col].min() >= 0 and values[col].quantile(0.99) >_
↳max(values[col].median() * 10, 10):
        values[col] = np.log1p(values[col])
        ylabel = f"log1p({col})"
    else:
        ylabel = col
    boxplot_print(values, target_col, col, ax=ax, showliers=False)
    ax.set_title(col)
    ax.set_ylabel(ylabel)
    ax.tick_params(axis="x", rotation=25)
for ax in axes[len(columns):]:
    ax.set_visible(False)
fig.suptitle(title, y=1.02)
plt.tight_layout()
plt.show()

```

0.1 Load Data

```

[2]: DATA_PATH = find_data_file("Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv")
raw_df = pd.read_csv(DATA_PATH, encoding="utf-8-sig")

# CICIDS headers contain leading spaces. Strip them so feature names are easier_
↳to use.
df = raw_df.copy()
df.columns = df.columns.str.strip()

numeric_cols_raw = df.select_dtypes(include=np.number).columns
inf_count = int(np.isinf(df[numeric_cols_raw].to_numpy()).sum())
print(f"Loaded from: {DATA_PATH}")
print(f"Infinite numeric values before cleanup: {inf_count}")

df[numeric_cols_raw] = df[numeric_cols_raw].replace([np.inf, -np.inf], np.nan)

```

Loaded from: Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
Infinite numeric values before cleanup: 64

```

[3]: TARGET_COL = 'Label'
EXCLUDE_COLS = ['Label']

```

```

PREFERRED_NUMERIC = ['Flow Duration', 'Total Fwd Packets', 'Total Backward_
↳Packets', 'Total Length of Fwd Packets', 'Total Length of Bwd Packets',
↳'Flow Bytes/s', 'Flow Packets/s', 'Packet Length Mean', 'Average Packet_
↳Size', 'Init_Win_bytes_forward', 'Init_Win_bytes_backward']
PREFERRED_CATEGORICAL = ['Label']

display_basic_profile(df, TARGET_COL)
display(df.head())

```

	metric	value
0	rows	225,745.0000
1	columns	79.0000
2	memory_mb	145.9400
3	duplicate_rows	2,633.0000
4	total_missing_cells	68.0000

	Label	count	percent
0	DDoS	128027	56.7100
1	BENIGN	97718	43.2900

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	\
0	54865	3	2		0
1	55054	109	1		1
2	55055	52	1		1
3	46236	34	1		1
4	54863	3	2		0

	Total Length of Fwd Packets	Total Length of Bwd Packets	\
0	12	0	
1	6	6	
2	6	6	
3	6	6	
4	12	0	

	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	\
0	6	6	6.0000	
1	6	6	6.0000	
2	6	6	6.0000	
3	6	6	6.0000	
4	6	6	6.0000	

	Fwd Packet Length Std	Bwd Packet Length Max	Bwd Packet Length Min	\
0	0.0000	0	0	
1	0.0000	6	6	
2	0.0000	6	6	
3	0.0000	6	6	
4	0.0000	0	0	

	Bwd Packet Length Mean	Bwd Packet Length Std	Flow Bytes/s	\
--	------------------------	-----------------------	--------------	---

0	0.0000	0.0000	4,000,000.0000
1	6.0000	0.0000	110,091.7431
2	6.0000	0.0000	230,769.2308
3	6.0000	0.0000	352,941.1765
4	0.0000	0.0000	4,000,000.0000

	Flow Packets/s	Flow IAT Mean	Flow IAT Std	Flow IAT Max	Flow IAT Min	\
0	666,666.6667	3.0000	0.0000	3	3	
1	18,348.6238	109.0000	0.0000	109	109	
2	38,461.5385	52.0000	0.0000	52	52	
3	58,823.5294	34.0000	0.0000	34	34	
4	666,666.6667	3.0000	0.0000	3	3	

	Fwd IAT Total	Fwd IAT Mean	Fwd IAT Std	Fwd IAT Max	Fwd IAT Min	\
0	3	3.0000	0.0000	3	3	
1	0	0.0000	0.0000	0	0	
2	0	0.0000	0.0000	0	0	
3	0	0.0000	0.0000	0	0	
4	3	3.0000	0.0000	3	3	

	Bwd IAT Total	Bwd IAT Mean	Bwd IAT Std	Bwd IAT Max	Bwd IAT Min	\
0	0	0.0000	0.0000	0	0	
1	0	0.0000	0.0000	0	0	
2	0	0.0000	0.0000	0	0	
3	0	0.0000	0.0000	0	0	
4	0	0.0000	0.0000	0	0	

	Fwd PSH Flags	Bwd PSH Flags	Fwd URG Flags	Bwd URG Flags	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	

	Fwd Header Length	Bwd Header Length	Fwd Packets/s	Bwd Packets/s	\
0	40	0	666,666.6667	0.0000	
1	20	20	9,174.3119	9,174.3119	
2	20	20	19,230.7692	19,230.7692	
3	20	20	29,411.7647	29,411.7647	
4	40	0	666,666.6667	0.0000	

	Min Packet Length	Max Packet Length	Packet Length Mean	\
0	6	6	6.0000	
1	6	6	6.0000	
2	6	6	6.0000	
3	6	6	6.0000	
4	6	6	6.0000	

	Packet Length Std	Packet Length Variance	FIN Flag Count	SYN Flag Count	\
0	0.0000	0.0000	0	0	
1	0.0000	0.0000	0	0	
2	0.0000	0.0000	0	0	
3	0.0000	0.0000	0	0	
4	0.0000	0.0000	0	0	

	RST Flag Count	PSH Flag Count	ACK Flag Count	URG Flag Count	\
0	0	0	1	0	
1	0	0	1	1	
2	0	0	1	1	
3	0	0	1	1	
4	0	0	1	0	

	CWE Flag Count	ECE Flag Count	Down/Up Ratio	Average Packet Size	\
0	0	0	0	9.0000	
1	0	0	1	9.0000	
2	0	0	1	9.0000	
3	0	0	1	9.0000	
4	0	0	0	9.0000	

	Avg Fwd Segment Size	Avg Bwd Segment Size	Fwd Header Length.1	\
0	6.0000	0.0000	40	
1	6.0000	6.0000	20	
2	6.0000	6.0000	20	
3	6.0000	6.0000	20	
4	6.0000	0.0000	40	

	Fwd Avg Bytes/Bulk	Fwd Avg Packets/Bulk	Fwd Avg Bulk Rate	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	Bwd Avg Bytes/Bulk	Bwd Avg Packets/Bulk	Bwd Avg Bulk Rate	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	Subflow Fwd Packets	Subflow Fwd Bytes	Subflow Bwd Packets	\
0	2	12	0	
1	1	6	1	
2	1	6	1	
3	1	6	1	
4	2	12	0	

	Subflow Bwd Bytes	Init_Win_bytes_forward	Init_Win_bytes_backward	\
0	0	33	-1	
1	6	29	256	
2	6	29	256	
3	6	31	329	
4	0	32	-1	

	act_data_pkt_fwd	min_seg_size_forward	Active Mean	Active Std	\
0	1	20	0.0000	0.0000	
1	0	20	0.0000	0.0000	
2	0	20	0.0000	0.0000	
3	0	20	0.0000	0.0000	
4	1	20	0.0000	0.0000	

	Active Max	Active Min	Idle Mean	Idle Std	Idle Max	Idle Min	Label
0	0	0	0.0000	0.0000	0	0	BENIGN
1	0	0	0.0000	0.0000	0	0	BENIGN
2	0	0	0.0000	0.0000	0	0	BENIGN
3	0	0	0.0000	0.0000	0	0	BENIGN
4	0	0	0.0000	0.0000	0	0	BENIGN

0.2 Data Quality

```
[4]: # Data quality checks
missing = (
    df.isna().sum()
    .rename("missing_count")
    .to_frame()
)
missing["missing_percent"] = (missing["missing_count"] / len(df) * 100).round(3)
missing = missing.sort_values("missing_count", ascending=False)

dtype_summary = (
    pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "non_null": df.notna().sum(),
        "missing": df.isna().sum(),
        "unique": df.nunique(dropna=False),
    })
    .sort_values(["missing", "unique"], ascending=[False, False])
)

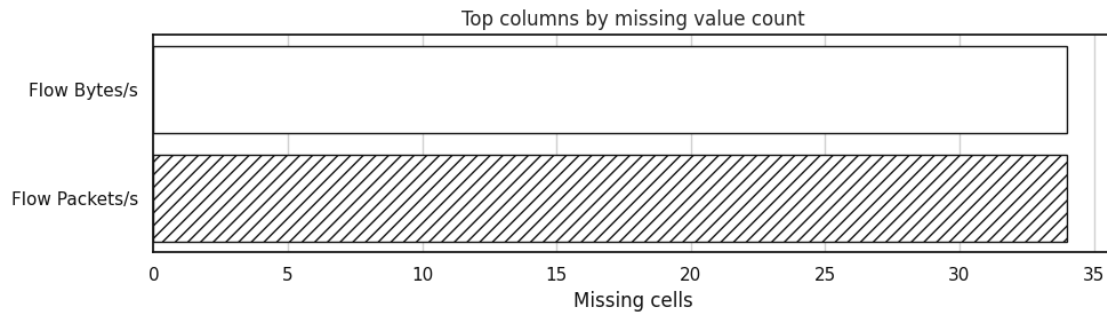
display(dtype_summary.head(30))
display(missing.head(20))
plot_missing_values(df)
```

dtype non_null missing unique

Flow Bytes/s	float64	225711	34	202293
Flow Packets/s	float64	225711	34	194094
Flow IAT Mean	float64	225745	0	193666
Fwd Packets/s	float64	225745	0	192230
Flow Duration	int64	225745	0	187752
Flow IAT Std	float64	225745	0	159622
Bwd Packets/s	float64	225745	0	144444
Flow IAT Max	int64	225745	0	139745
Bwd IAT Mean	float64	225745	0	117311
Bwd IAT Std	float64	225745	0	113859
Bwd IAT Total	int64	225745	0	101942
Bwd IAT Max	int64	225745	0	101379
Fwd IAT Mean	float64	225745	0	95095
Fwd IAT Std	float64	225745	0	91184
Fwd IAT Total	int64	225745	0	78049
Fwd IAT Max	int64	225745	0	77978
Idle Min	int64	225745	0	48018
Active Mean	float64	225745	0	40409
Active Max	int64	225745	0	40313
Active Min	int64	225745	0	38393
Idle Mean	float64	225745	0	35285
Idle Max	int64	225745	0	33002
Destination Port	int64	225745	0	23950
Packet Length Std	float64	225745	0	13502
Packet Length Variance	float64	225745	0	13502
Packet Length Mean	float64	225745	0	11680
Average Packet Size	float64	225745	0	11270
Flow IAT Min	int64	225745	0	11093
Bwd Packet Length Std	float64	225745	0	9650
Fwd Packet Length Std	float64	225745	0	9555

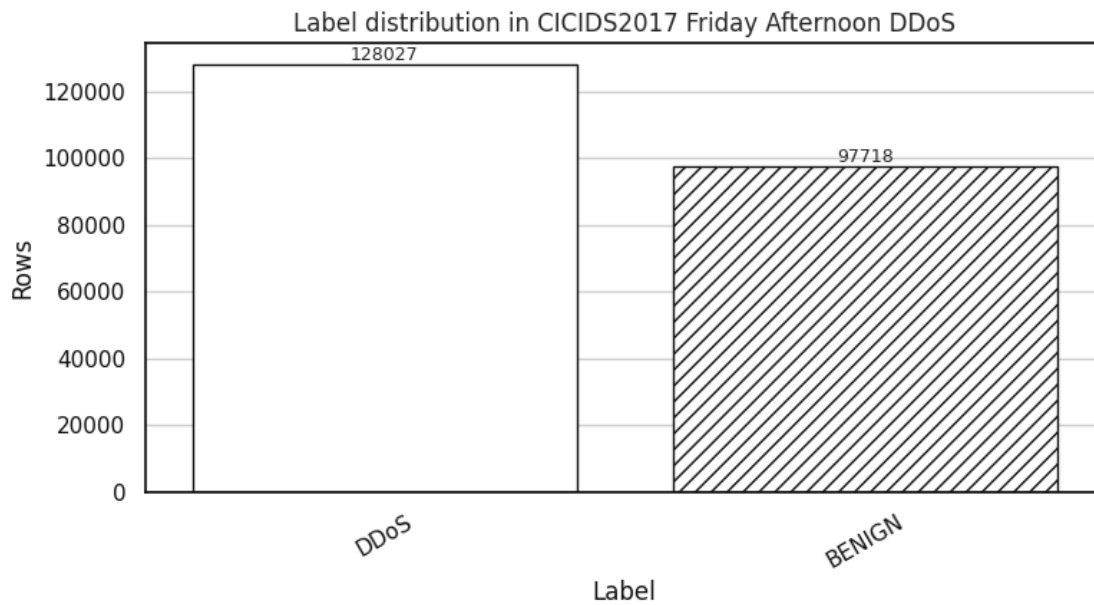
	missing_count	missing_percent
Flow Bytes/s	34	0.0150
Flow Packets/s	34	0.0150
Flow Duration	0	0.0000
Total Backward Packets	0	0.0000
Total Length of Fwd Packets	0	0.0000
Total Length of Bwd Packets	0	0.0000
Destination Port	0	0.0000
Fwd Packet Length Max	0	0.0000
Fwd Packet Length Min	0	0.0000
Fwd Packet Length Std	0	0.0000
Fwd Packet Length Mean	0	0.0000
Bwd Packet Length Max	0	0.0000
Bwd Packet Length Min	0	0.0000
Bwd Packet Length Mean	0	0.0000
Total Fwd Packets	0	0.0000
Bwd Packet Length Std	0	0.0000

Flow IAT Mean	0	0.0000
Flow IAT Std	0	0.0000
Flow IAT Max	0	0.0000
Flow IAT Min	0	0.0000



0.3 Target Distribution

```
[5]: plot_target_distribution(df, TARGET_COL, f"{TARGET_COL} distribution in_
      ↪CICIDS2017 Friday Afternoon DDoS")
```



0.4 Feature Summaries

```
[6]: # Numeric and categorical summaries
numeric_cols = get_numeric_columns(df, exclude=EXCLUDE_COLS)
categorical_cols = get_categorical_columns(df, exclude=EXCLUDE_COLS)

print(f"Numeric columns: {len(numeric_cols)}")
print(f"Categorical columns: {len(categorical_cols)}")

display(df[numeric_cols].describe(percentiles=[0.01, 0.05, 0.5, 0.95, 0.99]).T)
↳ if numeric_cols else pd.DataFrame()

categorical_profile = pd.DataFrame({
    "column": categorical_cols,
    "unique_values": [df[c].nunique(dropna=False) for c in categorical_cols],
    "top_value": [df[c].astype("string").fillna("<NA>").mode().iloc[0] if not
↳ df[c].mode(dropna=False).empty else None for c in categorical_cols],
    "top_count": [df[c].astype("string").fillna("<NA>").value_counts().iloc[0]
↳ if len(df) else 0 for c in categorical_cols],
})
display(categorical_profile.sort_values("unique_values", ascending=False))
```

Numeric columns: 78

Categorical columns: 0

	count	mean	std \
Destination Port	225,745.0000	8,879.6195	19,754.6474
Flow Duration	225,745.0000	16,241,648.5281	31,524,374.2319
Total Fwd Packets	225,745.0000	4.8749	15.4229
Total Backward Packets	225,745.0000	4.5728	21.7554
Total Length of Fwd Packets	225,745.0000	939.4633	3,249.4035
Total Length of Bwd Packets	225,745.0000	5,960.4775	39,218.3374
Fwd Packet Length Max	225,745.0000	538.5357	1,864.1290
Fwd Packet Length Min	225,745.0000	27.8822	163.3242
Fwd Packet Length Mean	225,745.0000	164.8267	504.8930
Fwd Packet Length Std	225,745.0000	214.9072	797.4111
Bwd Packet Length Max	225,745.0000	2,735.5851	3,705.1235
Bwd Packet Length Min	225,745.0000	16.7188	50.4806
Bwd Packet Length Mean	225,745.0000	890.5368	1,120.3249
Bwd Packet Length Std	225,745.0000	1,230.1729	1,733.2013
Flow Bytes/s	225,711.0000	585,393.8774	16,885,519.7337
Flow Packets/s	225,711.0000	14,241.2012	115,104.2398
Flow IAT Mean	225,745.0000	1,580,587.2374	2,701,595.7855
Flow IAT Std	225,745.0000	4,248,569.3807	7,622,819.0773
Flow IAT Max	225,745.0000	13,489,773.8237	26,701,716.6862
Flow IAT Min	225,745.0000	28,118.5477	759,810.0411
Fwd IAT Total	225,745.0000	15,396,522.9955	31,608,257.6648
Fwd IAT Mean	225,745.0000	2,540,609.5092	5,934,694.0016

Fwd IAT Std	225,745.0000	5,195,207.3809	10,786,352.4426
Fwd IAT Max	225,745.0000	12,994,339.1036	27,488,695.8151
Fwd IAT Min	225,745.0000	207,369.8267	3,795,227.5538
Bwd IAT Total	225,745.0000	6,564,701.0866	21,984,549.3313
Bwd IAT Mean	225,745.0000	947,632.2204	4,586,373.9761
Bwd IAT Std	225,745.0000	1,610,306.2432	5,475,777.6731
Bwd IAT Max	225,745.0000	4,567,514.4615	16,178,651.1935
Bwd IAT Min	225,745.0000	225,781.7441	4,019,289.7587
Fwd PSH Flags	225,745.0000	0.0332	0.1792
Bwd PSH Flags	225,745.0000	0.0000	0.0000
Fwd URG Flags	225,745.0000	0.0000	0.0000
Bwd URG Flags	225,745.0000	0.0000	0.0000
Fwd Header Length	225,745.0000	111.5227	375.7907
Bwd Header Length	225,745.0000	106.7890	511.7658
Fwd Packets/s	225,745.0000	12,615.0827	110,670.1357
Bwd Packets/s	225,745.0000	1,641.6927	19,895.9342
Min Packet Length	225,745.0000	8.0726	15.7677
Max Packet Length	225,745.0000	3,226.0453	3,813.1348
Packet Length Mean	225,745.0000	515.0021	559.0645
Packet Length Std	225,745.0000	1,085.5932	1,269.5587
Packet Length Variance	225,745.0000	2,789,905.5452	4,115,940.9628
FIN Flag Count	225,745.0000	0.0027	0.0516
SYN Flag Count	225,745.0000	0.0332	0.1792
RST Flag Count	225,745.0000	0.0001	0.0109
PSH Flag Count	225,745.0000	0.3512	0.4773
ACK Flag Count	225,745.0000	0.5045	0.5000
URG Flag Count	225,745.0000	0.1408	0.3478
CWE Flag Count	225,745.0000	0.0000	0.0000
ECE Flag Count	225,745.0000	0.0001	0.0109
Down/Up Ratio	225,745.0000	1.0058	1.4308
Average Packet Size	225,745.0000	574.5688	626.0962
Avg Fwd Segment Size	225,745.0000	164.8267	504.8930
Avg Bwd Segment Size	225,745.0000	890.5368	1,120.3249
Fwd Header Length.1	225,745.0000	111.5227	375.7907
Fwd Avg Bytes/Bulk	225,745.0000	0.0000	0.0000
Fwd Avg Packets/Bulk	225,745.0000	0.0000	0.0000
Fwd Avg Bulk Rate	225,745.0000	0.0000	0.0000
Bwd Avg Bytes/Bulk	225,745.0000	0.0000	0.0000
Bwd Avg Packets/Bulk	225,745.0000	0.0000	0.0000
Bwd Avg Bulk Rate	225,745.0000	0.0000	0.0000
Subflow Fwd Packets	225,745.0000	4.8749	15.4229
Subflow Fwd Bytes	225,745.0000	939.4633	3,249.4035
Subflow Bwd Packets	225,745.0000	4.5728	21.7554
Subflow Bwd Bytes	225,745.0000	5,960.4775	39,218.3374
Init_Win_bytes_forward	225,745.0000	4,247.4369	8,037.7810
Init_Win_bytes_backward	225,745.0000	601.0486	4,319.7203
act_data_pkt_fwd	225,745.0000	3.3115	12.2700
min_seg_size_forward	225,745.0000	21.4828	4.1668

Active Mean	225,745.0000	184,826.0927	797,925.0418
Active Std	225,745.0000	12,934.3576	210,273.6700
Active Max	225,745.0000	208,084.8686	900,234.9870
Active Min	225,745.0000	177,620.0889	784,260.2205
Idle Mean	225,745.0000	10,322,143.4188	21,853,028.1491
Idle Std	225,745.0000	3,611,942.9274	12,756,893.4583
Idle Max	225,745.0000	12,878,128.9439	26,921,263.6458
Idle Min	225,745.0000	7,755,355.0929	19,831,094.4505

	min	1%	5%	50% \
Destination Port	0.0000	53.0000	53.0000	80.0000
Flow Duration	-1.0000	4.0000	161.0000	1,452,333.0000
Total Fwd Packets	1.0000	1.0000	1.0000	3.0000
Total Backward Packets	0.0000	0.0000	0.0000	4.0000
Total Length of Fwd Packets	0.0000	0.0000	6.0000	30.0000
Total Length of Bwd Packets	0.0000	0.0000	0.0000	164.0000
Fwd Packet Length Max	0.0000	0.0000	6.0000	20.0000
Fwd Packet Length Min	0.0000	0.0000	0.0000	0.0000
Fwd Packet Length Mean	0.0000	0.0000	6.0000	8.6667
Fwd Packet Length Std	0.0000	0.0000	0.0000	5.3020
Bwd Packet Length Max	0.0000	0.0000	0.0000	99.0000
Bwd Packet Length Min	0.0000	0.0000	0.0000	0.0000
Bwd Packet Length Mean	0.0000	0.0000	0.0000	92.0000
Bwd Packet Length Std	0.0000	0.0000	0.0000	2.4495
Flow Bytes/s	-12,000,000.0000	0.0000	2.4847	1,133.4656
Flow Packets/s	-2,000,000.0000	0.1286	0.1531	5.1754
Flow IAT Mean	-1.0000	3.0000	61.0000	224,516.8571
Flow IAT Std	0.0000	0.0000	0.0000	564,167.6299
Flow IAT Max	-1.0000	4.0000	124.0000	1,422,624.0000
Flow IAT Min	-12.0000	0.0000	1.0000	3.0000
Fwd IAT Total	0.0000	0.0000	0.0000	23,710.0000
Fwd IAT Mean	0.0000	0.0000	0.0000	10,329.3333
Fwd IAT Std	0.0000	0.0000	0.0000	12,734.9931
Fwd IAT Max	0.0000	0.0000	0.0000	23,028.0000
Fwd IAT Min	-12.0000	0.0000	0.0000	4.0000
Bwd IAT Total	0.0000	0.0000	0.0000	41,101.0000
Bwd IAT Mean	0.0000	0.0000	0.0000	10,009.4000
Bwd IAT Std	0.0000	0.0000	0.0000	14,703.7189
Bwd IAT Max	0.0000	0.0000	0.0000	33,365.0000
Bwd IAT Min	0.0000	0.0000	0.0000	3.0000
Fwd PSH Flags	0.0000	0.0000	0.0000	0.0000
Bwd PSH Flags	0.0000	0.0000	0.0000	0.0000
Fwd URG Flags	0.0000	0.0000	0.0000	0.0000
Bwd URG Flags	0.0000	0.0000	0.0000	0.0000
Fwd Header Length	0.0000	20.0000	20.0000	72.0000
Bwd Header Length	0.0000	0.0000	0.0000	92.0000
Fwd Packets/s	0.0000	0.0598	0.0827	2.3156
Bwd Packets/s	0.0000	0.0000	0.0000	1.4833

Min Packet Length	0.0000	0.0000	0.0000	0.0000
Max Packet Length	0.0000	0.0000	6.0000	513.0000
Packet Length Mean	0.0000	0.0000	6.0000	110.3333
Packet Length Std	0.0000	0.0000	0.0000	154.0308
Packet Length Variance	0.0000	0.0000	0.0000	23,725.4909
FIN Flag Count	0.0000	0.0000	0.0000	0.0000
SYN Flag Count	0.0000	0.0000	0.0000	0.0000
RST Flag Count	0.0000	0.0000	0.0000	0.0000
PSH Flag Count	0.0000	0.0000	0.0000	0.0000
ACK Flag Count	0.0000	0.0000	0.0000	1.0000
URG Flag Count	0.0000	0.0000	0.0000	0.0000
CWE Flag Count	0.0000	0.0000	0.0000	0.0000
ECE Flag Count	0.0000	0.0000	0.0000	0.0000
Down/Up Ratio	0.0000	0.0000	0.0000	1.0000
Average Packet Size	0.0000	0.0000	6.8571	141.0000
Avg Fwd Segment Size	0.0000	0.0000	6.0000	8.6667
Avg Bwd Segment Size	0.0000	0.0000	0.0000	92.0000
Fwd Header Length.1	0.0000	20.0000	20.0000	72.0000
Fwd Avg Bytes/Bulk	0.0000	0.0000	0.0000	0.0000
Fwd Avg Packets/Bulk	0.0000	0.0000	0.0000	0.0000
Fwd Avg Bulk Rate	0.0000	0.0000	0.0000	0.0000
Bwd Avg Bytes/Bulk	0.0000	0.0000	0.0000	0.0000
Bwd Avg Packets/Bulk	0.0000	0.0000	0.0000	0.0000
Bwd Avg Bulk Rate	0.0000	0.0000	0.0000	0.0000
Subflow Fwd Packets	1.0000	1.0000	1.0000	3.0000
Subflow Fwd Bytes	0.0000	0.0000	6.0000	30.0000
Subflow Bwd Packets	0.0000	0.0000	0.0000	4.0000
Subflow Bwd Bytes	0.0000	0.0000	0.0000	164.0000
Init_Win_bytes_forward	-1.0000	-1.0000	-1.0000	256.0000
Init_Win_bytes_backward	-1.0000	-1.0000	-1.0000	126.0000
act_data_pkt_fwd	0.0000	0.0000	0.0000	2.0000
min_seg_size_forward	0.0000	20.0000	20.0000	20.0000
Active Mean	0.0000	0.0000	0.0000	0.0000
Active Std	0.0000	0.0000	0.0000	0.0000
Active Max	0.0000	0.0000	0.0000	0.0000
Active Min	0.0000	0.0000	0.0000	0.0000
Idle Mean	0.0000	0.0000	0.0000	0.0000
Idle Std	0.0000	0.0000	0.0000	0.0000
Idle Max	0.0000	0.0000	0.0000	0.0000
Idle Min	0.0000	0.0000	0.0000	0.0000

	95%	99% \
Destination Port	58,731.8000	63,808.5600
Flow Duration	93,204,499.8000	115,860,163.7600
Total Fwd Packets	9.0000	27.0000
Total Backward Packets	9.0000	29.0000
Total Length of Fwd Packets	11,607.0000	11,613.0000
Total Length of Bwd Packets	11,607.0000	17,432.5600

Fwd Packet Length Max	5,755.0000	10,135.0000
Fwd Packet Length Min	47.0000	1,375.0000
Fwd Packet Length Mean	1,658.1429	2,321.4000
Fwd Packet Length Std	2,127.5977	4,059.4619
Bwd Packet Length Max	10,135.0000	11,595.0000
Bwd Packet Length Min	114.0000	185.0000
Bwd Packet Length Mean	2,900.2500	2,901.7500
Bwd Packet Length Std	4,413.2020	5,795.5007
Flow Bytes/s	1,205,992.5090	4,000,000.0000
Flow Packets/s	22,857.1429	600,000.0000
Flow IAT Mean	7,069,856.3456	8,466,438.4165
Flow IAT Std	23,000,000.0000	27,000,000.0000
Flow IAT Max	81,500,000.0000	94,700,000.0000
Flow IAT Min	14,851.8000	149,502.4400
Fwd IAT Total	92,700,000.0000	116,000,000.0000
Fwd IAT Mean	13,700,000.0000	20,200,000.0000
Fwd IAT Std	32,900,000.0000	40,800,000.0000
Fwd IAT Max	83,200,000.0000	96,800,000.0000
Fwd IAT Min	992.0000	192,434.0400
Bwd IAT Total	72,700,000.0000	111,000,000.0000
Bwd IAT Mean	6,643,371.6710	11,900,000.0000
Bwd IAT Std	5,447,657.3180	30,000,000.0000
Bwd IAT Max	12,100,000.0000	87,700,000.0000
Bwd IAT Min	209.0000	17,013.8000
Fwd PSH Flags	0.0000	1.0000
Bwd PSH Flags	0.0000	0.0000
Fwd URG Flags	0.0000	0.0000
Bwd URG Flags	0.0000	0.0000
Fwd Header Length	192.0000	712.0000
Bwd Header Length	204.0000	744.0000
Fwd Packets/s	11,904.7619	532,133.3334
Bwd Packets/s	9,950.2488	19,607.8431
Min Packet Length	44.0000	58.0000
Max Packet Length	10,220.0000	11,595.0000
Packet Length Mean	1,453.3750	1,454.1250
Packet Length Std	3,350.6349	4,097.5375
Packet Length Variance	11,200,000.0000	16,800,000.0000
FIN Flag Count	0.0000	0.0000
SYN Flag Count	0.0000	1.0000
RST Flag Count	0.0000	0.0000
PSH Flag Count	1.0000	1.0000
ACK Flag Count	1.0000	1.0000
URG Flag Count	1.0000	1.0000
CWE Flag Count	0.0000	0.0000
ECE Flag Count	0.0000	0.0000
Down/Up Ratio	5.0000	6.0000
Average Packet Size	1,661.0000	1,661.8571
Avg Fwd Segment Size	1,658.1429	2,321.4000

Avg Bwd Segment Size	2,900.2500	2,901.7500
Fwd Header Length.1	192.0000	712.0000
Fwd Avg Bytes/Bulk	0.0000	0.0000
Fwd Avg Packets/Bulk	0.0000	0.0000
Fwd Avg Bulk Rate	0.0000	0.0000
Bwd Avg Bytes/Bulk	0.0000	0.0000
Bwd Avg Packets/Bulk	0.0000	0.0000
Bwd Avg Bulk Rate	0.0000	0.0000
Subflow Fwd Packets	9.0000	27.0000
Subflow Fwd Bytes	11,607.0000	11,613.0000
Subflow Bwd Packets	9.0000	29.0000
Subflow Bwd Bytes	11,607.0000	17,432.5600
Init_Win_bytes_forward	8,192.0000	30,228.0000
Init_Win_bytes_backward	258.0000	16,563.9200
act_data_pkt_fwd	7.0000	21.0000
min_seg_size_forward	32.0000	40.0000
Active Mean	1,470,603.4666	4,134,663.0800
Active Std	0.0000	248,292.4872
Active Max	1,822,956.4000	4,205,998.6400
Active Min	1,377,968.8000	4,125,898.8800
Idle Mean	71,500,000.0000	93,956,000.0000
Idle Std	43,600,000.0000	57,100,000.0000
Idle Max	81,500,000.0000	94,700,000.0000
Idle Min	71,500,000.0000	93,956,000.0000

	max
Destination Port	65,532.0000
Flow Duration	119,999,937.0000
Total Fwd Packets	1,932.0000
Total Backward Packets	2,942.0000
Total Length of Fwd Packets	183,012.0000
Total Length of Bwd Packets	5,172,346.0000
Fwd Packet Length Max	11,680.0000
Fwd Packet Length Min	1,472.0000
Fwd Packet Length Mean	3,867.0000
Fwd Packet Length Std	6,692.6450
Bwd Packet Length Max	11,680.0000
Bwd Packet Length Min	1,460.0000
Bwd Packet Length Mean	5,800.5000
Bwd Packet Length Std	8,194.6605
Flow Bytes/s	2,070,000,000.0000
Flow Packets/s	3,000,000.0000
Flow IAT Mean	107,000,000.0000
Flow IAT Std	69,200,000.0000
Flow IAT Max	120,000,000.0000
Flow IAT Min	107,000,000.0000
Fwd IAT Total	120,000,000.0000
Fwd IAT Mean	120,000,000.0000

Fwd IAT Std	76,700,000.0000
Fwd IAT Max	120,000,000.0000
Fwd IAT Min	120,000,000.0000
Bwd IAT Total	120,000,000.0000
Bwd IAT Mean	120,000,000.0000
Bwd IAT Std	76,700,000.0000
Bwd IAT Max	120,000,000.0000
Bwd IAT Min	120,000,000.0000
Fwd PSH Flags	1.0000
Bwd PSH Flags	0.0000
Fwd URG Flags	0.0000
Bwd URG Flags	0.0000
Fwd Header Length	39,396.0000
Bwd Header Length	58,852.0000
Fwd Packets/s	3,000,000.0000
Bwd Packets/s	2,000,000.0000
Min Packet Length	337.0000
Max Packet Length	11,680.0000
Packet Length Mean	1,936.8333
Packet Length Std	4,731.5224
Packet Length Variance	22,400,000.0000
FIN Flag Count	1.0000
SYN Flag Count	1.0000
RST Flag Count	1.0000
PSH Flag Count	1.0000
ACK Flag Count	1.0000
URG Flag Count	1.0000
CWE Flag Count	0.0000
ECE Flag Count	1.0000
Down/Up Ratio	7.0000
Average Packet Size	2,528.0000
Avg Fwd Segment Size	3,867.0000
Avg Bwd Segment Size	5,800.5000
Fwd Header Length.1	39,396.0000
Fwd Avg Bytes/Bulk	0.0000
Fwd Avg Packets/Bulk	0.0000
Fwd Avg Bulk Rate	0.0000
Bwd Avg Bytes/Bulk	0.0000
Bwd Avg Packets/Bulk	0.0000
Bwd Avg Bulk Rate	0.0000
Subflow Fwd Packets	1,932.0000
Subflow Fwd Bytes	183,012.0000
Subflow Bwd Packets	2,942.0000
Subflow Bwd Bytes	5,172,346.0000
Init_Win_bytes_forward	65,535.0000
Init_Win_bytes_backward	65,535.0000
act_data_pkt_fwd	1,931.0000
min_seg_size_forward	52.0000

```

Active Mean      100,000,000.0000
Active Std       39,500,000.0000
Active Max       100,000,000.0000
Active Min       100,000,000.0000
Idle Mean        120,000,000.0000
Idle Std         65,300,000.0000
Idle Max         120,000,000.0000
Idle Min         120,000,000.0000

```

Empty DataFrame

Columns: [column, unique_values, top_value, top_count]

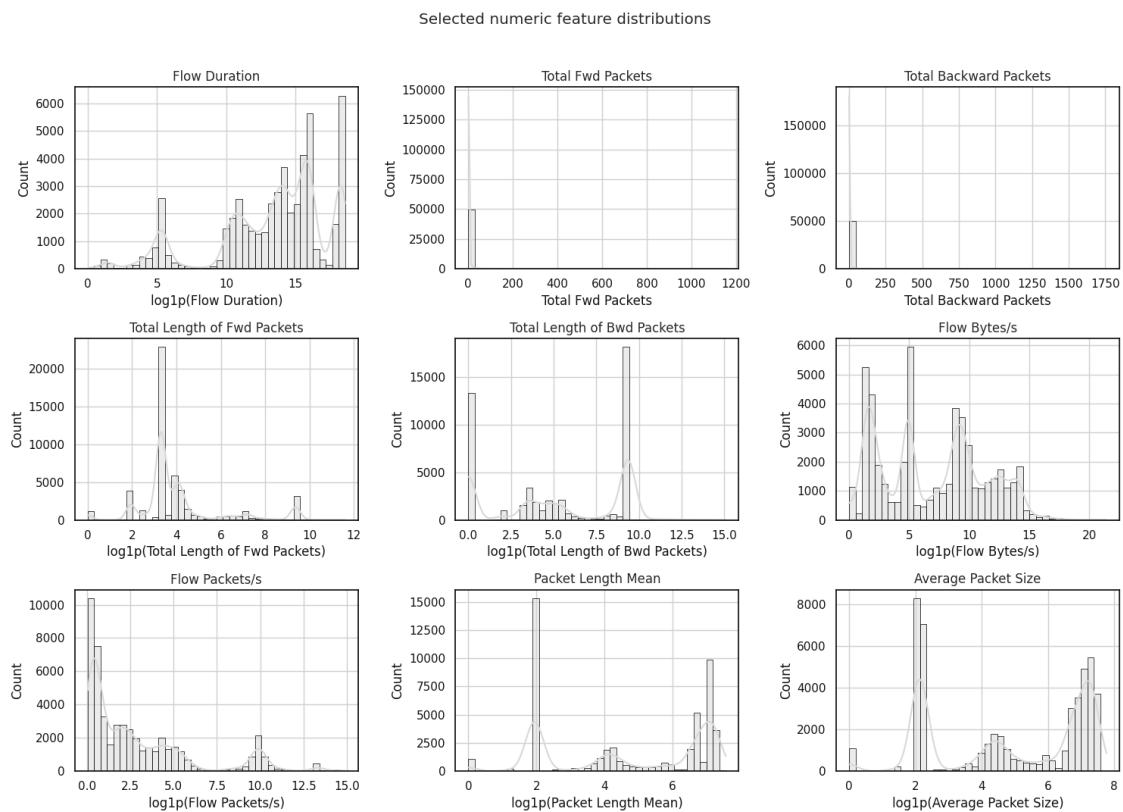
Index: []

0.5 Univariate Patterns

```

[7]: plot_numeric_distributions(df, [c for c in PREFERRED_NUMERIC if c in ↵
    ↵numeric_cols][:9], "Selected numeric feature distributions")
plot_categorical_top_counts(df, [c for c in PREFERRED_CATEGORICAL if c in ↵
    ↵categorical_cols][:6], "Top category counts")

```



No categorical columns available for count plots.

0.6 Target Relationships

```
[8]: # Target associations and correlation structure
assoc = target_association(df, TARGET_COL, numeric_cols, exclude=EXCLUDE_COLS)
display(assoc.head(20).rename("absolute_correlation_with_target_code").
        ↪to_frame())

selected_numeric = assoc.head(18).index.tolist()
if len(selected_numeric) < 2:
    selected_numeric = numeric_cols[:18]

plot_correlation_heatmap(df, selected_numeric, "Correlation heatmap for_
    ↪selected numeric features")
plot_numeric_by_target(df, TARGET_COL, selected_numeric[:6], "Selected numeric_
    ↪features by target")

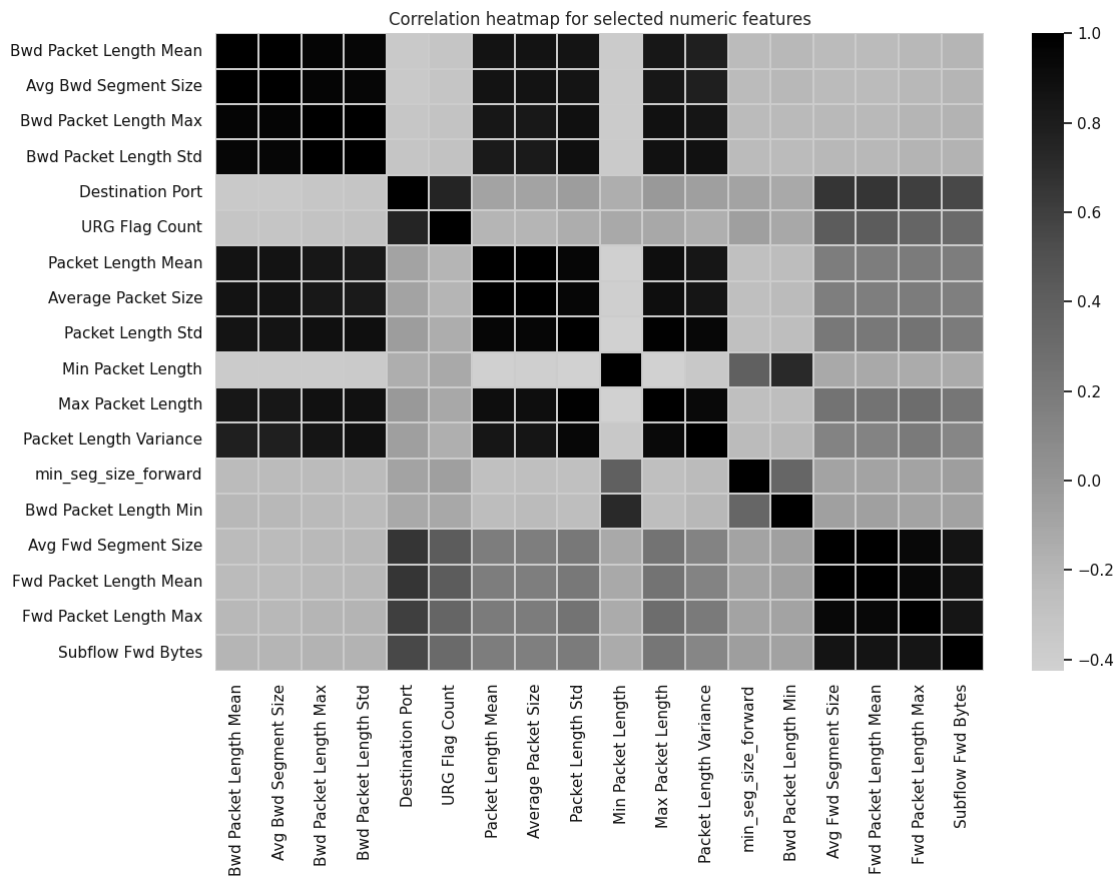
if selected_numeric:
    grouped_median = df.groupby(TARGET_COL)[selected_numeric[:10]].
    ↪median(numeric_only=True).T
    display(grouped_median)
```

```
/home/durgaumadev/security-for-data-science-lab/.venv/lib/python3.13/site-
packages/numpy/lib/_function_base_impl.py:3023: RuntimeWarning: invalid value
encountered in divide
  c /= stddev[:, None]
/home/durgaumadev/security-for-data-science-lab/.venv/lib/python3.13/site-
packages/numpy/lib/_function_base_impl.py:3024: RuntimeWarning: invalid value
encountered in divide
  c /= stddev[None, :]
```

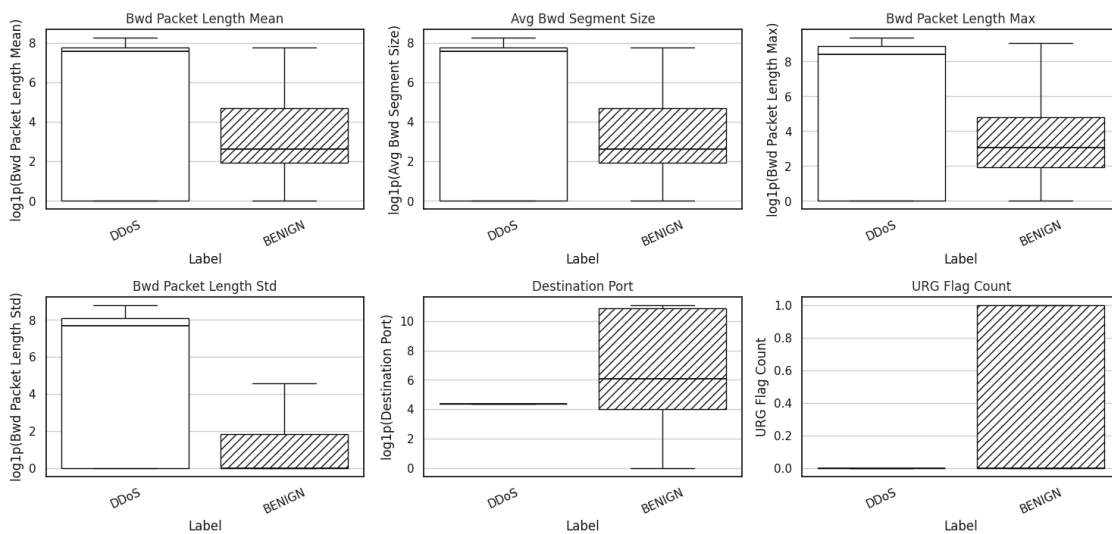
	absolute_correlation_with_target_code
Bwd Packet Length Mean	0.6033
Avg Bwd Segment Size	0.6033
Bwd Packet Length Max	0.5773
Bwd Packet Length Std	0.5762
Destination Port	0.5098
URG Flag Count	0.4632
Packet Length Mean	0.4543
Average Packet Size	0.4535
Packet Length Std	0.4437
Min Packet Length	0.4274
Max Packet Length	0.4144
Packet Length Variance	0.4081
min_seg_size_forward	0.4073
Bwd Packet Length Min	0.3657
Avg Fwd Segment Size	0.3569
Fwd Packet Length Mean	0.3569
Fwd Packet Length Max	0.3215

Subflow Fwd Bytes
 Total Length of Fwd Packets
 Bwd IAT Total

0.3197
 0.3197
 0.3187



Selected numeric features by target



Label	BENIGN	DDoS
Bwd Packet Length Mean	13.0000	1,934.5000
Avg Bwd Segment Size	13.0000	1,934.5000
Bwd Packet Length Max	20.0000	5,755.0000
Bwd Packet Length Std	0.0000	2,189.7729
Destination Port	443.0000	80.0000
URG Flag Count	0.0000	0.0000
Packet Length Mean	61.0000	833.5000
Average Packet Size	77.5000	897.6154
Packet Length Std	32.2047	1,903.9627
Min Packet Length	6.0000	0.0000

0.7 Advanced EDA

This section extends the baseline EDA with DDoS-focused model-preparation checks: column-name cleanup, class imbalance, duplicate rows, missing and infinite values, zero-dominant features, outliers, skewness, correlated features, low-variance signals, PCA separation, mutual-information feature ranking, and final preprocessing recommendations.

```
[9]: from sklearn.decomposition import PCA
from sklearn.feature_selection import mutual_info_classif
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

ADVANCED_SAMPLE_SIZE = 12_000
FEATURE_RANK_SAMPLE_SIZE = 60_000
CORRELATION_THRESHOLD = 0.85
ZERO_DOMINANT_THRESHOLD = 0.90
DOMINANT_VALUE_THRESHOLD = 0.95
LOW_VARIANCE_THRESHOLD = 1e-8

def clean_numeric_frame(data, columns):
    if not columns:
        return pd.DataFrame(index=data.index)
    return data[columns].apply(pd.to_numeric, errors="coerce").replace([np.inf,
↪ -np.inf], np.nan)

def count_table(data, col):
    counts = data[col].astype("string").fillna("<NA>").
↪ value_counts(dropna=False)
    table = counts.rename_axis(col).reset_index(name="count")
    table["percent"] = (table["count"] / len(data) * 100).round(3)
    table["ratio_vs_minority"] = (table["count"] / table["count"].min()).
↪ round(3)
    return table
```



```

def plot_horizontal_counts(table, label_col, value_col="count", title="Count_
↳plot", color="white"):
    if table.empty:
        display(Markdown("No rows available for plotting."))
        return
    plt.figure(figsize=(9, max(3, 0.35 * len(table))))
    ax = sns.barplot(data=table, x=value_col, y=label_col, color="white",
↳edgecolor="#111111")
    apply_bar_hatches(ax)
    plt.title(title)
    plt.xlabel(value_col.replace("_", " ").title())
    plt.ylabel("")
    plt.tight_layout()
    plt.show()

def safe_inf_counts(numeric_data):
    if numeric_data.empty:
        return {"pos_inf": 0, "neg_inf": 0, "total_inf": 0}
    arr = numeric_data.to_numpy(dtype="float64", copy=False)
    pos_inf = int(np.isposinf(arr).sum())
    neg_inf = int(np.isneginf(arr).sum())
    return {"pos_inf": pos_inf, "neg_inf": neg_inf, "total_inf": pos_inf +
↳neg_inf}

def pca_projection(data, numeric_columns, color_col, sample_size, title):
    pca_numeric_cols = [
        col for col in numeric_columns
        if data[col].nunique(dropna=True) > 1 and pd.to_numeric(data[col],
↳errors="coerce").notna().any()
    ]
    if len(pca_numeric_cols) < 2:
        display(Markdown("PCA requires at least two non-constant numeric_
↳features."))
        return pd.DataFrame()

    pca_sample = sampled(data[[color_col] + pca_numeric_cols], sample_size)
    X_numeric = clean_numeric_frame(pca_sample, pca_numeric_cols)
    X_imputed = SimpleImputer(strategy="median").fit_transform(X_numeric)
    X_scaled = StandardScaler().fit_transform(X_imputed)
    pca = PCA(n_components=2, random_state=RANDOM_STATE)
    pcs = pca.fit_transform(X_scaled)
    pca_result = pd.DataFrame({
        "PC1": pcs[:, 0],
        "PC2": pcs[:, 1],
        color_col: pca_sample[color_col].astype("string").fillna("<NA>").
↳to_numpy(),

```

```

}))
explained = pca.explained_variance_ratio_ * 100
display(pd.DataFrame({
    "component": ["PC1", "PC2"],
    "explained_variance_percent": explained.round(3),
}))

scatter_with_markers(
    pca_result,
    "PC1",
    "PC2",
    color_col,
    title,
    xlabel=f"PC1 ({explained[0]:.2f}% variance)",
    ylabel=f"PC2 ({explained[1]:.2f}% variance)",
)
return pca_result

```

0.8 Column Name Cleanup Check

```

[10]: raw_columns = pd.Index(raw_df.columns)
clean_columns = raw_columns.str.strip()
column_cleanup_table = pd.DataFrame({
    "raw_column": raw_columns,
    "clean_column": clean_columns,
    "changed_by_strip": raw_columns != clean_columns,
})

cleanup_summary = pd.DataFrame({
    "metric": ["columns_changed_by_strip", "duplicate_columns_after_strip", ↵
↵ "current_duplicate_columns"],
    "value": [
        int(column_cleanup_table["changed_by_strip"].sum()),
        int(clean_columns.duplicated().sum()),
        int(pd.Index(df.columns).duplicated().sum()),
    ],
})
display(cleanup_summary)
display(column_cleanup_table[column_cleanup_table["changed_by_strip"]].head(30))

```

	metric	value
0	columns_changed_by_strip	65
1	duplicate_columns_after_strip	0
2	current_duplicate_columns	0

	raw_column	clean_column \
0	Destination Port	Destination Port
1	Flow Duration	Flow Duration

2	Total Fwd Packets	Total Fwd Packets
3	Total Backward Packets	Total Backward Packets
5	Total Length of Bwd Packets	Total Length of Bwd Packets
6	Fwd Packet Length Max	Fwd Packet Length Max
7	Fwd Packet Length Min	Fwd Packet Length Min
8	Fwd Packet Length Mean	Fwd Packet Length Mean
9	Fwd Packet Length Std	Fwd Packet Length Std
11	Bwd Packet Length Min	Bwd Packet Length Min
12	Bwd Packet Length Mean	Bwd Packet Length Mean
13	Bwd Packet Length Std	Bwd Packet Length Std
15	Flow Packets/s	Flow Packets/s
16	Flow IAT Mean	Flow IAT Mean
17	Flow IAT Std	Flow IAT Std
18	Flow IAT Max	Flow IAT Max
19	Flow IAT Min	Flow IAT Min
21	Fwd IAT Mean	Fwd IAT Mean
22	Fwd IAT Std	Fwd IAT Std
23	Fwd IAT Max	Fwd IAT Max
24	Fwd IAT Min	Fwd IAT Min
26	Bwd IAT Mean	Bwd IAT Mean
27	Bwd IAT Std	Bwd IAT Std
28	Bwd IAT Max	Bwd IAT Max
29	Bwd IAT Min	Bwd IAT Min
31	Bwd PSH Flags	Bwd PSH Flags
32	Fwd URG Flags	Fwd URG Flags
33	Bwd URG Flags	Bwd URG Flags
34	Fwd Header Length	Fwd Header Length
35	Bwd Header Length	Bwd Header Length

	changed_by_strip
0	True
1	True
2	True
3	True
5	True
6	True
7	True
8	True
9	True
11	True
12	True
13	True
15	True
16	True
17	True
18	True
19	True
21	True

```

22         True
23         True
24         True
26         True
27         True
28         True
29         True
31         True
32         True
33         True
34         True
35         True

```

0.9 Class Imbalance Analysis For Label

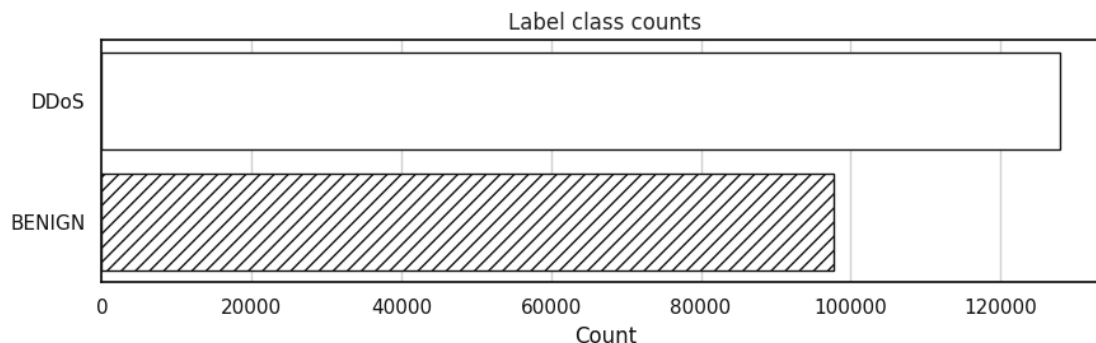
```

[11]: label_balance = count_table(df, TARGET_COL)
display(label_balance)
plot_horizontal_counts(label_balance, TARGET_COL, "count", "Label class_
↪counts", color="white")

if len(label_balance) == 2:
    majority = label_balance.iloc[0]
    minority = label_balance.iloc[-1]
    display(Markdown(
        f"Binary target detected. Majority `{majority[TARGET_COL]}` has_
↪{majority['count'],} rows; "
        f"minority `{minority[TARGET_COL]}` has {minority['count'],} rows.
↪Majority/minority ratio: "
        f"{majority['ratio_vs_minority']:.3f}:1."
    ))
else:
    display(Markdown(f"`{TARGET_COL}` has {len(label_balance),} classes."))

```

	Label	count	percent	ratio_vs_minority
0	DDoS	128027	56.7130	1.3100
1	BENIGN	97718	43.2870	1.0000



Binary target detected. Majority DDoS has 128,027 rows; minority BENIGN has 97,718 rows. Majority/minority ratio: 1.310:1.

0.10 Duplicate Analysis

```
[12]: duplicate_mask = df.duplicated(keep=False)
duplicate_rows = df[duplicate_mask].copy()

duplicate_summary = pd.DataFrame({
    "metric": ["duplicate_rows_keep_first", "rows_in_duplicate_groups",
    ↪ "duplicate_groups"],
    "value": [
        int(df.duplicated().sum()),
        int(duplicate_mask.sum()),
        int(df[duplicate_mask].value_counts(dropna=False).shape[0]) if
    ↪ duplicate_mask.any() else 0,
    ],
})
display(duplicate_summary)

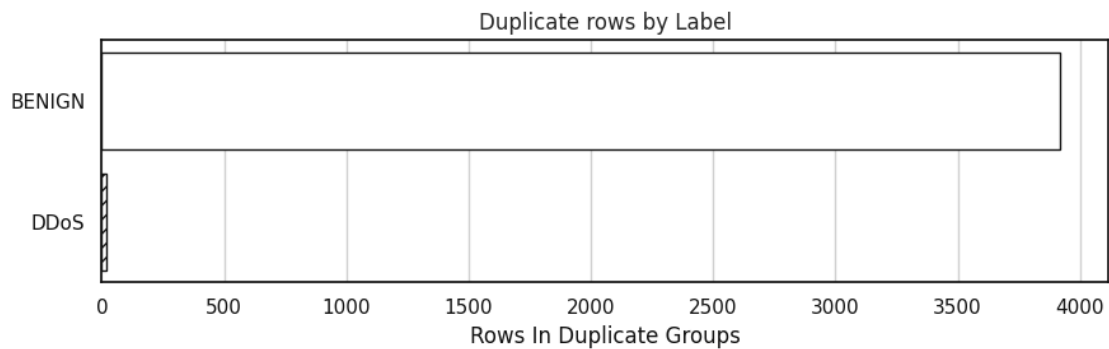
if duplicate_mask.any():
    duplicate_by_label = (
        duplicate_rows[TARGET_COL]
        .astype("string")
        .fillna("<NA>")
        .value_counts()
        .rename_axis(TARGET_COL)
        .reset_index(name="rows_in_duplicate_groups")
    )
    duplicate_by_label["percent_of_duplicate_rows"] =
    ↪ (duplicate_by_label["rows_in_duplicate_groups"] / len(duplicate_rows) * 100).
    ↪ round(3)
    display(duplicate_by_label)
    plot_horizontal_counts(duplicate_by_label, TARGET_COL,
    ↪ "rows_in_duplicate_groups", "Duplicate rows by Label", color="white")

    duplicate_group_sizes = (
        df[duplicate_mask]
        .groupby(df.columns.tolist(), dropna=False)
        .size()
        .reset_index(name="duplicate_group_size")
        .sort_values("duplicate_group_size", ascending=False)
    )
    display(duplicate_group_sizes[["duplicate_group_size", TARGET_COL] + [c for
    ↪ c in PREFERRED_NUMERIC if c in duplicate_group_sizes.columns][:6]].head(10))
```

```
else:
    display(Markdown("No duplicate rows found."))
```

	metric	value
0	duplicate_rows_keep_first	2633
1	rows_in_duplicate_groups	3940
2	duplicate_groups	1307

	Label	rows_in_duplicate_groups	percent_of_duplicate_rows
0	BENIGN	3920	99.4920
1	DDoS	20	0.5080



	duplicate_group_size	Label	Flow Duration	Total Fwd Packets \
1190	118	BENIGN	3	2
1187	42	BENIGN	1	2
1199	32	BENIGN	4	2
1204	27	BENIGN	48	2
807	20	BENIGN	247	2
1139	18	BENIGN	3	2
1207	18	BENIGN	49	2
1142	17	BENIGN	4	2
1140	15	BENIGN	3	2
845	13	BENIGN	251	2

	Total Backward Packets	Total Length of Fwd Packets \
1190	0	12
1187	0	12
1199	0	12
1204	0	12
807	2	64
1139	0	124
1207	0	12
1142	0	124
1140	0	124
845	2	64

	Total Length of Bwd Packets	Flow Bytes/s
1190	0	4,000,000.0000
1187	0	12,000,000.0000
1199	0	3,000,000.0000
1204	0	250,000.0000
807	258	1,303,643.7250
1139	0	41,300,000.0000
1207	0	244,897.9592
1142	0	31,000,000.0000
1140	0	41,300,000.0000
845	258	1,282,868.5260

0.11 Missing, Inf, And -Inf Handling Analysis

```
[13]: raw_numeric = raw_df.select_dtypes(include=np.number)
raw_inf_counts = safe_inf_counts(raw_numeric)
current_numeric = clean_numeric_frame(df, numeric_cols)
current_inf_counts = safe_inf_counts(df[numeric_cols].select_dtypes(include=np.
↪number))

invalid_summary = pd.DataFrame({
    "metric": [
        "raw_positive_inf_values",
        "raw_negative_inf_values",
        "raw_total_inf_values",
        "current_positive_inf_values",
        "current_negative_inf_values",
        "current_total_inf_values",
        "current_missing_cells",
    ],
    "value": [
        raw_inf_counts["pos_inf"],
        raw_inf_counts["neg_inf"],
        raw_inf_counts["total_inf"],
        current_inf_counts["pos_inf"],
        current_inf_counts["neg_inf"],
        current_inf_counts["total_inf"],
        int(df.isna().sum().sum()),
    ],
})
display(invalid_summary)

raw_inf_by_col = pd.DataFrame(index=raw_numeric.columns.str.strip())
if not raw_numeric.empty:
    raw_arr = raw_numeric.to_numpy(dtype="float64", copy=False)
    raw_inf_by_col["raw_positive_inf"] = np.isposinf(raw_arr).sum(axis=0)
```

```

    raw_inf_by_col["raw_negative_inf"] = np.isneginf(raw_arr).sum(axis=0)
    raw_inf_by_col["raw_total_inf"] = raw_inf_by_col["raw_positive_inf"] +
    ↪raw_inf_by_col["raw_negative_inf"]
else:
    raw_inf_by_col["raw_positive_inf"] = []
    raw_inf_by_col["raw_negative_inf"] = []
    raw_inf_by_col["raw_total_inf"] = []

quality_table = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "missing_count": df.isna().sum(),
    "missing_percent": (df.isna().sum() / len(df) * 100).round(3),
    "unique_values": df.nunique(dropna=False),
})
quality_table = quality_table.join(raw_inf_by_col, how="left").fillna({
    "raw_positive_inf": 0,
    "raw_negative_inf": 0,
    "raw_total_inf": 0,
})
quality_table[["raw_positive_inf", "raw_negative_inf", "raw_total_inf"]] =
    ↪quality_table[["raw_positive_inf", "raw_negative_inf", "raw_total_inf"]].
    ↪astype(int)
quality_table = quality_table.sort_values(["missing_count", "raw_total_inf"],
    ↪ascending=False)
display(quality_table.head(30))

plot_missing_values(df)

```

	metric	value
0	raw_positive_inf_values	64
1	raw_negative_inf_values	0
2	raw_total_inf_values	64
3	current_positive_inf_values	0
4	current_negative_inf_values	0
5	current_total_inf_values	0
6	current_missing_cells	68

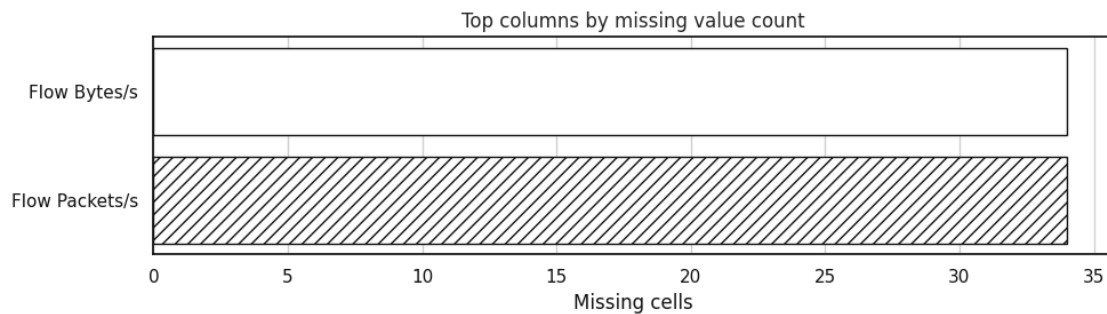
	dtype	missing_count	missing_percent	\
Flow Packets/s	float64	34	0.0150	
Flow Bytes/s	float64	34	0.0150	
Destination Port	int64	0	0.0000	
Flow Duration	int64	0	0.0000	
Total Fwd Packets	int64	0	0.0000	
Total Backward Packets	int64	0	0.0000	
Total Length of Fwd Packets	int64	0	0.0000	
Total Length of Bwd Packets	int64	0	0.0000	
Fwd Packet Length Max	int64	0	0.0000	
Fwd Packet Length Min	int64	0	0.0000	

Fwd Packet Length Mean	float64	0	0.0000
Fwd Packet Length Std	float64	0	0.0000
Bwd Packet Length Max	int64	0	0.0000
Bwd Packet Length Min	int64	0	0.0000
Bwd Packet Length Mean	float64	0	0.0000
Bwd Packet Length Std	float64	0	0.0000
Flow IAT Mean	float64	0	0.0000
Flow IAT Std	float64	0	0.0000
Flow IAT Max	int64	0	0.0000
Flow IAT Min	int64	0	0.0000
Fwd IAT Total	int64	0	0.0000
Fwd IAT Mean	float64	0	0.0000
Fwd IAT Std	float64	0	0.0000
Fwd IAT Max	int64	0	0.0000
Fwd IAT Min	int64	0	0.0000
Bwd IAT Total	int64	0	0.0000
Bwd IAT Mean	float64	0	0.0000
Bwd IAT Std	float64	0	0.0000
Bwd IAT Max	int64	0	0.0000
Bwd IAT Min	int64	0	0.0000

	unique_values	raw_positive_inf \
Flow Packets/s	194094	34
Flow Bytes/s	202293	30
Destination Port	23950	0
Flow Duration	187752	0
Total Fwd Packets	297	0
Total Backward Packets	367	0
Total Length of Fwd Packets	3831	0
Total Length of Bwd Packets	6760	0
Fwd Packet Length Max	1891	0
Fwd Packet Length Min	151	0
Fwd Packet Length Mean	7401	0
Fwd Packet Length Std	9555	0
Bwd Packet Length Max	1945	0
Bwd Packet Length Min	343	0
Bwd Packet Length Mean	8655	0
Bwd Packet Length Std	9650	0
Flow IAT Mean	193666	0
Flow IAT Std	159622	0
Flow IAT Max	139745	0
Flow IAT Min	11093	0
Fwd IAT Total	78049	0
Fwd IAT Mean	95095	0
Fwd IAT Std	91184	0
Fwd IAT Max	77978	0
Fwd IAT Min	8820	0
Bwd IAT Total	101942	0

Bwd IAT Mean	117311	0
Bwd IAT Std	113859	0
Bwd IAT Max	101379	0
Bwd IAT Min	3787	0

	raw_negative_inf	raw_total_inf
Flow Packets/s	0	34
Flow Bytes/s	0	30
Destination Port	0	0
Flow Duration	0	0
Total Fwd Packets	0	0
Total Backward Packets	0	0
Total Length of Fwd Packets	0	0
Total Length of Bwd Packets	0	0
Fwd Packet Length Max	0	0
Fwd Packet Length Min	0	0
Fwd Packet Length Mean	0	0
Fwd Packet Length Std	0	0
Bwd Packet Length Max	0	0
Bwd Packet Length Min	0	0
Bwd Packet Length Mean	0	0
Bwd Packet Length Std	0	0
Flow IAT Mean	0	0
Flow IAT Std	0	0
Flow IAT Max	0	0
Flow IAT Min	0	0
Fwd IAT Total	0	0
Fwd IAT Mean	0	0
Fwd IAT Std	0	0
Fwd IAT Max	0	0
Fwd IAT Min	0	0
Bwd IAT Total	0	0
Bwd IAT Mean	0	0
Bwd IAT Std	0	0
Bwd IAT Max	0	0
Bwd IAT Min	0	0



0.12 Zero-Dominant Feature Analysis

```
[14]: numeric_for_zero = clean_numeric_frame(df, numeric_cols)
zero_counts = numeric_for_zero.eq(0).sum()
zero_dominance_table = pd.DataFrame({
    "column": zero_counts.index,
    "zero_count": zero_counts.astype(int).values,
    "zero_percent": (zero_counts / len(df) * 100).round(3).values,
    "non_missing_count": numeric_for_zero.notna().sum().astype(int).values,
    "unique_values": [df[c].nunique(dropna=False) for c in zero_counts.index],
}).sort_values("zero_percent", ascending=False)
zero_dominance_table["is_zero_dominant"] = zero_dominance_table["zero_percent"]
    >= ZERO_DOMINANT_THRESHOLD * 100

display(zero_dominance_table.head(30))
zero_dominant_features =
    zero_dominance_table[zero_dominance_table["is_zero_dominant"]]
display(zero_dominant_features)

if not zero_dominant_features.empty:
    plot_horizontal_counts(zero_dominant_features.head(20), "column",
        "zero_percent", "Zero-dominant numeric features", color="white")
else:
    display(Markdown(f"No numeric features have at least
        {ZERO_DOMINANT_THRESHOLD:.0%} zero values."))
```

	column	zero_count	zero_percent	non_missing_count	\
61	Bwd Avg Bulk Rate	225745	100.0000	225745	
60	Bwd Avg Packets/Bulk	225745	100.0000	225745	
58	Fwd Avg Bulk Rate	225745	100.0000	225745	
31	Bwd PSH Flags	225745	100.0000	225745	
56	Fwd Avg Bytes/Bulk	225745	100.0000	225745	
57	Fwd Avg Packets/Bulk	225745	100.0000	225745	
49	CWE Flag Count	225745	100.0000	225745	
59	Bwd Avg Bytes/Bulk	225745	100.0000	225745	
33	Bwd URG Flags	225745	100.0000	225745	
32	Fwd URG Flags	225745	100.0000	225745	
45	RST Flag Count	225718	99.9880	225745	
50	ECE Flag Count	225718	99.9880	225745	
43	FIN Flag Count	225142	99.7330	225745	
71	Active Std	220059	97.4810	225745	
30	Fwd PSH Flags	218245	96.6780	225745	
44	SYN Flag Count	218245	96.6780	225745	
75	Idle Std	203206	90.0160	225745	
48	URG Flag Count	193971	85.9250	225745	
11	Bwd Packet Length Min	167203	74.0670	225745	

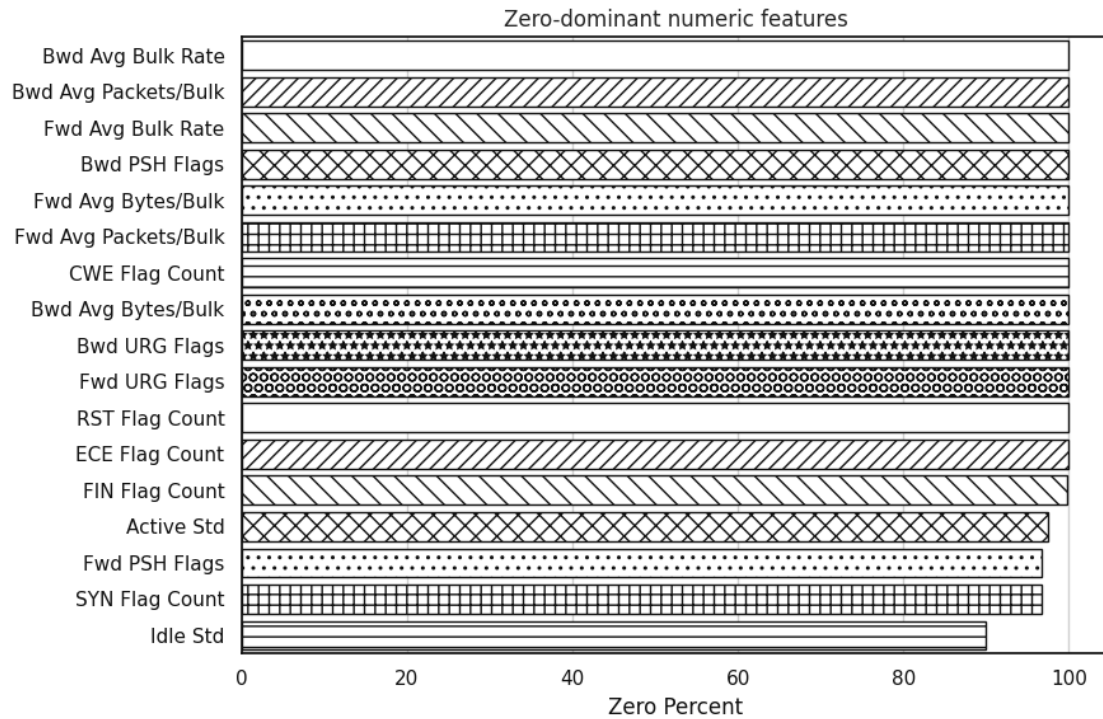
46	PSH Flag Count	146472	64.8840	225745
70	Active Mean	145540	64.4710	225745
72	Active Max	145540	64.4710	225745
73	Active Min	145540	64.4710	225745
76	Idle Max	145481	64.4450	225745
77	Idle Min	145481	64.4450	225745
74	Idle Mean	145481	64.4450	225745
38	Min Packet Length	120861	53.5390	225745
7	Fwd Packet Length Min	117256	51.9420	225745
47	ACK Flag Count	111865	49.5540	225745
13	Bwd Packet Length Std	111583	49.4290	225745

	unique_values	is_zero_dominant
61	1	True
60	1	True
58	1	True
31	1	True
56	1	True
57	1	True
49	1	True
59	1	True
33	1	True
32	1	True
45	2	True
50	2	True
43	2	True
71	5669	True
30	2	True
44	2	True
75	5857	True
48	2	False
11	343	False
46	2	False
70	40409	False
72	40313	False
73	38393	False
76	33002	False
77	48018	False
74	35285	False
38	109	False
7	151	False
47	2	False
13	9650	False

	column	zero_count	zero_percent	non_missing_count	\
61	Bwd Avg Bulk Rate	225745	100.0000	225745	
60	Bwd Avg Packets/Bulk	225745	100.0000	225745	
58	Fwd Avg Bulk Rate	225745	100.0000	225745	

31	Bwd PSH Flags	225745	100.0000	225745
56	Fwd Avg Bytes/Bulk	225745	100.0000	225745
57	Fwd Avg Packets/Bulk	225745	100.0000	225745
49	CWE Flag Count	225745	100.0000	225745
59	Bwd Avg Bytes/Bulk	225745	100.0000	225745
33	Bwd URG Flags	225745	100.0000	225745
32	Fwd URG Flags	225745	100.0000	225745
45	RST Flag Count	225718	99.9880	225745
50	ECE Flag Count	225718	99.9880	225745
43	FIN Flag Count	225142	99.7330	225745
71	Active Std	220059	97.4810	225745
30	Fwd PSH Flags	218245	96.6780	225745
44	SYN Flag Count	218245	96.6780	225745
75	Idle Std	203206	90.0160	225745

	unique_values	is_zero_dominant
61	1	True
60	1	True
58	1	True
31	1	True
56	1	True
57	1	True
49	1	True
59	1	True
33	1	True
32	1	True
45	2	True
50	2	True
43	2	True
71	5669	True
30	2	True
44	2	True
75	5857	True



0.13 IQR-Based Outlier Count Table

```
[15]: outlier_rows = []
for col in numeric_cols:
    values = pd.to_numeric(df[col], errors="coerce").replace([np.inf, -np.inf], np.nan).dropna()
    if values.empty:
        continue
    q1 = values.quantile(0.25)
    q3 = values.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    if np.isclose(iqr, 0):
        outlier_count = 0
    else:
        outlier_count = int(((values < lower) | (values > upper)).sum())
    outlier_rows.append({
        "column": col,
        "q1": q1,
        "q3": q3,
        "iqr": iqr,
        "lower_bound": lower,
```

```

        "upper_bound": upper,
        "outlier_count": outlier_count,
        "outlier_percent": round(outlier_count / len(values) * 100, 3),
        "non_null": int(values.shape[0]),
    })

iqr_outlier_table = pd.DataFrame(outlier_rows).sort_values("outlier_count",
    ↪ascending=False)
display(iqr_outlier_table.head(30))

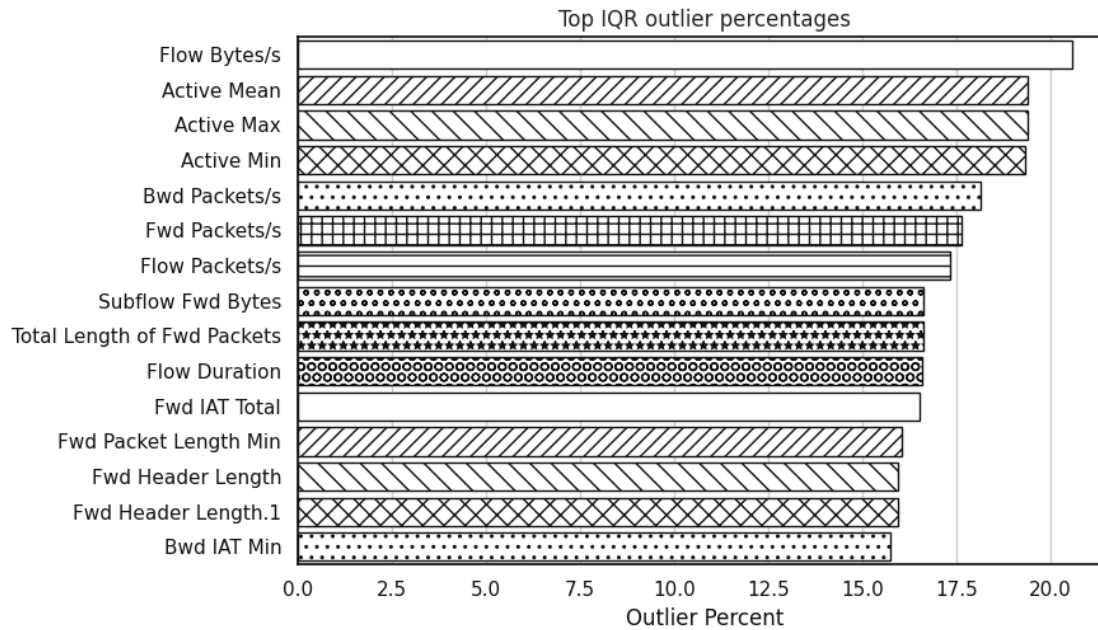
plot_iqr = iqr_outlier_table[iqr_outlier_table["outlier_count"] > 0].head(15)
if not plot_iqr.empty:
    plot_horizontal_counts(plot_iqr, "column", "outlier_percent", "Top IQR",
    ↪outlier percentages", color="white")
else:
    display(Markdown("No IQR outliers detected after excluding zero-IQR columns.
    ↪"))

```

	column	q1	q3	iqr \
14	Flow Bytes/s	12.0819	21,580.2368	21,568.1549
70	Active Mean	0.0000	1,878.0000	1,878.0000
72	Active Max	0.0000	1,878.0000	1,878.0000
73	Active Min	0.0000	1,862.0000	1,862.0000
37	Bwd Packets/s	0.0353	21.0247	20.9895
36	Fwd Packets/s	0.4247	32.8727	32.4481
15	Flow Packets/s	0.6185	70.3561	69.7376
63	Subflow Fwd Bytes	26.0000	63.0000	37.0000
4	Total Length of Fwd Packets	26.0000	63.0000	37.0000
1	Flow Duration	71,180.0000	8,805,237.0000	8,734,057.0000
20	Fwd IAT Total	129.0000	7,869,855.0000	7,869,726.0000
7	Fwd Packet Length Min	0.0000	6.0000	6.0000
34	Fwd Header Length	64.0000	100.0000	36.0000
55	Fwd Header Length.1	64.0000	100.0000	36.0000
29	Bwd IAT Min	0.0000	47.0000	47.0000
21	Fwd IAT Mean	110.0000	2,017,643.0000	2,017,533.0000
11	Bwd Packet Length Min	0.0000	6.0000	6.0000
18	Flow IAT Max	61,178.0000	8,358,910.0000	8,297,732.0000
76	Idle Max	0.0000	8,253,838.0000	8,253,838.0000
74	Idle Mean	0.0000	8,239,725.0000	8,239,725.0000
6	Fwd Packet Length Max	6.0000	34.0000	28.0000
38	Min Packet Length	0.0000	6.0000	6.0000
23	Fwd IAT Max	124.0000	7,548,056.0000	7,547,932.0000
17	Flow IAT Std	19,104.4627	4,033,232.4370	4,014,127.9743
19	Flow IAT Min	1.0000	39.0000	38.0000
22	Fwd IAT Std	0.0000	3,680,321.0900	3,680,321.0900
25	Bwd IAT Total	0.0000	1,111,677.0000	1,111,677.0000
28	Bwd IAT Max	0.0000	1,094,045.0000	1,094,045.0000

16	Flow IAT Mean	19,181.6250	2,013,458.6670	1,994,277.0420
26	Bwd IAT Mean	0.0000	265,583.7500	265,583.7500

	lower_bound	upper_bound	outlier_count	outlier_percent	non_null
14	-32,340.1505	53,932.4693	46394	20.5550	225711
70	-2,817.0000	4,695.0000	43738	19.3750	225745
72	-2,817.0000	4,695.0000	43738	19.3750	225745
73	-2,793.0000	4,655.0000	43648	19.3350	225745
37	-31.4489	52.5090	40923	18.1280	225745
36	-48.2474	81.5448	39806	17.6330	225745
15	-103.9880	174.9626	39133	17.3380	225711
63	-29.5000	118.5000	37478	16.6020	225745
4	-29.5000	118.5000	37478	16.6020	225745
1	-13,029,905.5000	21,906,322.5000	37465	16.5960	225745
20	-11,804,460.0000	19,674,444.0000	37265	16.5080	225745
7	-9.0000	15.0000	36226	16.0470	225745
34	10.0000	154.0000	35999	15.9470	225745
55	10.0000	154.0000	35999	15.9470	225745
29	-70.5000	117.5000	35504	15.7270	225745
21	-3,026,189.5000	5,043,942.5000	34552	15.3060	225745
11	-9.0000	15.0000	33758	14.9540	225745
18	-12,385,420.0000	20,805,508.0000	33333	14.7660	225745
76	-12,380,757.0000	20,634,595.0000	33100	14.6630	225745
74	-12,359,587.5000	20,599,312.5000	33057	14.6440	225745
6	-36.0000	76.0000	33009	14.6220	225745
38	-9.0000	15.0000	32927	14.5860	225745
23	-11,321,774.0000	18,869,954.0000	32890	14.5700	225745
17	-6,002,087.4988	10,054,424.3985	32468	14.3830	225745
19	-56.0000	96.0000	32148	14.2410	225745
22	-5,520,481.6350	9,200,802.7250	31945	14.1510	225745
25	-1,667,515.5000	2,779,192.5000	31473	13.9420	225745
28	-1,641,067.5000	2,735,112.5000	31246	13.8410	225745
16	-2,972,233.9380	5,004,874.2300	30789	13.6390	225745
26	-398,375.6250	663,959.3750	30429	13.4790	225745



0.14 Skewness Ranking And Log-Transform Suggestions

```
[16]: skew_values = clean_numeric_frame(df, numeric_cols).skew(numeric_only=True).
      ↪dropna()
skewness_table = (
    skew_values.rename("skewness")
    .to_frame()
    .assign(abs_skewness=lambda x: x["skewness"].abs())
    .sort_values("abs_skewness", ascending=False)
)

log_transform_suggestions = []
for col in skewness_table.index:
    values = pd.to_numeric(df[col], errors="coerce").replace([np.inf, -np.inf],
    ↪np.nan).dropna()
    if values.empty:
        suggestion = "skip_empty"
    elif values.min() >= 0 and abs(skewness_table.loc[col, "skewness"]) >= 2:
        suggestion = "consider_log1p"
    elif values.min() > 0 and abs(skewness_table.loc[col, "skewness"]) >= 2:
        suggestion = "consider_log"
    elif abs(skewness_table.loc[col, "skewness"]) >= 2:
        suggestion = "consider_robust_or_quantile_transform"
    else:
        suggestion = "no_log_transform_needed"
    log_transform_suggestions.append(suggestion)
```

```

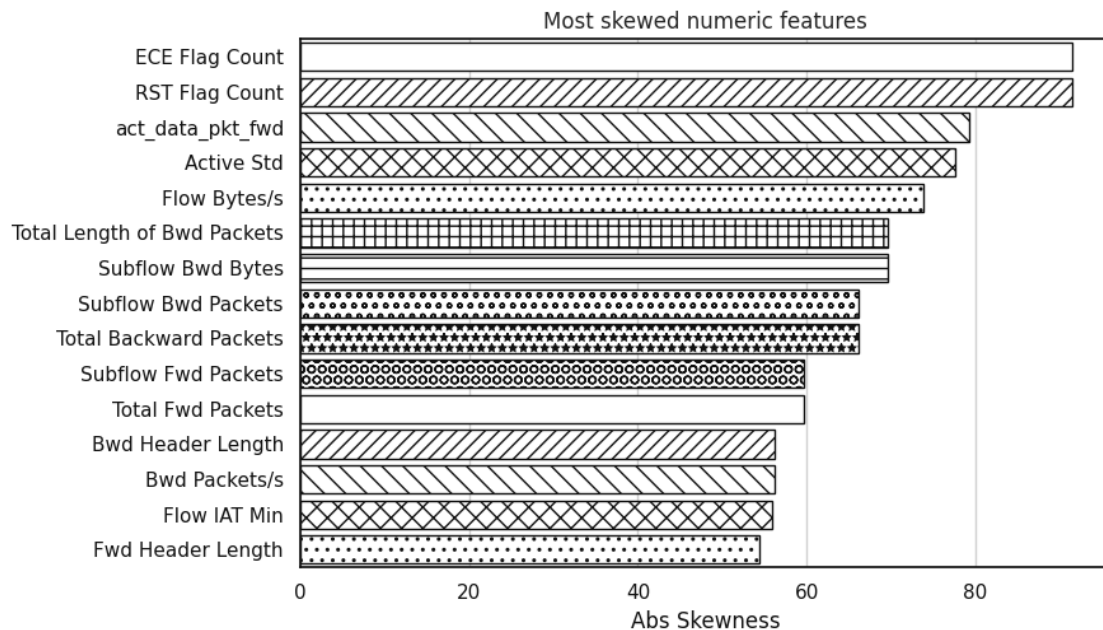
skewness_table["transform_suggestion"] = log_transform_suggestions
display(skewness_table.head(30))

plot_skew = skewness_table.head(15).reset_index(names="column")
if not plot_skew.empty:
    plot_horizontal_counts(plot_skew, "column", "abs_skewness", "Most skewed_
↳numeric features", color="white")

```

	skewness	abs_skewness	\
ECE Flag Count	91.4223	91.4223	
RST Flag Count	91.4223	91.4223	
act_data_pkt_fwd	79.2606	79.2606	
Active Std	77.5387	77.5387	
Flow Bytes/s	73.8468	73.8468	
Total Length of Bwd Packets	69.5972	69.5972	
Subflow Bwd Bytes	69.5972	69.5972	
Subflow Bwd Packets	66.2118	66.2118	
Total Backward Packets	66.2118	66.2118	
Subflow Fwd Packets	59.6370	59.6370	
Total Fwd Packets	59.6370	59.6370	
Bwd Header Length	56.2763	56.2763	
Bwd Packets/s	56.1751	56.1751	
Flow IAT Min	55.9233	55.9233	
Fwd Header Length	54.4109	54.4109	
Fwd Header Length.1	54.4109	54.4109	
Fwd IAT Min	22.6884	22.6884	
Bwd IAT Min	20.7737	20.7737	
FIN Flag Count	19.2712	19.2712	
Active Min	15.4437	15.4437	
Active Mean	15.1860	15.1860	
Active Max	15.1373	15.1373	
Bwd IAT Mean	14.2141	14.2141	
Fwd Packets/s	13.3307	13.3307	
Flow Packets/s	13.0352	13.0352	
Init_Win_bytes_backward	11.3942	11.3942	
Bwd Packet Length Min	10.4333	10.4333	
Fwd Packet Length Min	8.0406	8.0406	
Total Length of Fwd Packets	7.5223	7.5223	
Subflow Fwd Bytes	7.5223	7.5223	
		transform_suggestion	
ECE Flag Count		consider_log1p	
RST Flag Count		consider_log1p	
act_data_pkt_fwd		consider_log1p	
Active Std		consider_log1p	
Flow Bytes/s	consider_robust_or_quantile_transform		
Total Length of Bwd Packets		consider_log1p	

Subflow Bwd Bytes	consider_log1p
Subflow Bwd Packets	consider_log1p
Total Backward Packets	consider_log1p
Subflow Fwd Packets	consider_log1p
Total Fwd Packets	consider_log1p
Bwd Header Length	consider_log1p
Bwd Packets/s	consider_log1p
Flow IAT Min	consider_robust_or_quantile_transform
Fwd Header Length	consider_log1p
Fwd Header Length.1	consider_log1p
Fwd IAT Min	consider_robust_or_quantile_transform
Bwd IAT Min	consider_log1p
FIN Flag Count	consider_log1p
Active Min	consider_log1p
Active Mean	consider_log1p
Active Max	consider_log1p
Bwd IAT Mean	consider_log1p
Fwd Packets/s	consider_log1p
Flow Packets/s	consider_robust_or_quantile_transform
Init_Win_bytes_backward	consider_robust_or_quantile_transform
Bwd Packet Length Min	consider_log1p
Fwd Packet Length Min	consider_log1p
Total Length of Fwd Packets	consider_log1p
Subflow Fwd Bytes	consider_log1p



0.15 High-Correlation Feature Pair Detection

```
[17]: numeric_clean = clean_numeric_frame(df, numeric_cols)
corr_matrix = numeric_clean.corr().abs()
upper_mask = np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
correlated_pairs = (
    corr_matrix.where(upper_mask)
    .stack()
    .rename("absolute_correlation")
    .reset_index()
    .rename(columns={"level_0": "feature_1", "level_1": "feature_2"})
    .sort_values("absolute_correlation", ascending=False)
)
highly_correlated_pairs =
    correlated_pairs[correlated_pairs["absolute_correlation"] >=
    CORRELATION_THRESHOLD]
display(highly_correlated_pairs.head(40))
display(Markdown(f"Found {len(highly_correlated_pairs):,} numeric feature pairs,
    with absolute correlation >= {CORRELATION_THRESHOLD:.2f}."))
```

	feature_1	feature_2 \
677	Fwd Packet Length Mean	Avg Fwd Segment Size
3560	RST Flag Count	ECE Flag Count
990	Bwd Packet Length Mean	Avg Bwd Segment Size
298	Total Backward Packets	Subflow Bwd Packets
218	Total Fwd Packets	Subflow Fwd Packets
2707	Fwd Header Length	Fwd Header Length.1
375	Total Length of Fwd Packets	Subflow Fwd Bytes
2384	Fwd PSH Flags	SYN Flag Count
455	Total Length of Bwd Packets	Subflow Bwd Bytes
3172	Packet Length Mean	Average Packet Size
1480	Flow IAT Max	Idle Max
98	Flow Duration	Fwd IAT Total
1427	Flow IAT Max	Fwd IAT Max
1870	Fwd IAT Max	Idle Max
793	Bwd Packet Length Max	Bwd Packet Length Std
477	Fwd Packet Length Max	Fwd Packet Length Std
3083	Max Packet Length	Packet Length Std
1206	Flow Packets/s	Fwd Packets/s
5533	Active Mean	Active Min
1739	Fwd IAT Std	Fwd IAT Max
1344	Flow IAT Std	Flow IAT Max
1402	Flow IAT Std	Idle Max
269	Total Backward Packets	Bwd Header Length
2794	Bwd Header Length	Subflow Bwd Packets
1426	Flow IAT Max	Fwd IAT Std
454	Total Length of Bwd Packets	Subflow Bwd Packets
299	Total Backward Packets	Subflow Bwd Bytes

5057	Subflow Bwd Packets	Subflow Bwd Bytes
239	Total Backward Packets	Total Length of Bwd Packets
1792	Fwd IAT Std	Idle Max
1349	Flow IAT Std	Fwd IAT Max
211	Total Fwd Packets	Fwd Header Length.1
190	Total Fwd Packets	Fwd Header Length
2714	Fwd Header Length	Subflow Fwd Packets
4352	Fwd Header Length.1	Subflow Fwd Packets
792	Bwd Packet Length Max	Bwd Packet Length Mean
834	Bwd Packet Length Max	Avg Bwd Segment Size
2134	Bwd IAT Std	Bwd IAT Max
159	Total Fwd Packets	Total Backward Packets
4900	Subflow Fwd Packets	Subflow Bwd Packets

	absolute_correlation
677	1.0000
3560	1.0000
990	1.0000
298	1.0000
218	1.0000
2707	1.0000
375	1.0000
2384	1.0000
455	1.0000
3172	0.9994
1480	0.9973
98	0.9971
1427	0.9953
1870	0.9930
793	0.9927
477	0.9917
3083	0.9840
1206	0.9838
5533	0.9827
1739	0.9805
1344	0.9780
1402	0.9758
269	0.9756
2794	0.9756
1426	0.9716
454	0.9704
299	0.9704
5057	0.9704
239	0.9704
1792	0.9698
1349	0.9693
211	0.9684
190	0.9684

2714	0.9684
4352	0.9684
792	0.9614
834	0.9614
2134	0.9586
159	0.9567
4900	0.9567

Found 122 numeric feature pairs with absolute correlation ≥ 0.85 .

0.16 Low-Variance Feature Detection

```
[18]: low_variance_rows = []
for col in [c for c in df.columns if c != TARGET_COL]:
    series = df[col]
    value_counts = series.astype("string").fillna("<NA>").
    ↪value_counts(dropna=False)
    dominant_percent = float(value_counts.iloc[0] / len(series) * 100) if
    ↪len(value_counts) else np.nan
    unique_values = int(series.nunique(dropna=False))
    row = {
        "column": col,
        "dtype": str(series.dtype),
        "unique_values": unique_values,
        "is_constant": unique_values <= 1,
        "dominant_value": value_counts.index[0] if len(value_counts) else None,
        "dominant_percent": round(dominant_percent, 3),
        "near_constant_by_dominance": dominant_percent >=
    ↪DOMINANT_VALUE_THRESHOLD * 100 if len(value_counts) else False,
    }
    if pd.api.types.is_numeric_dtype(series):
        values = pd.to_numeric(series, errors="coerce").replace([np.inf, -np.
    ↪inf], np.nan)
        row["variance"] = float(values.var(skipna=True)) if values.notna().
    ↪any() else np.nan
        row["zero_or_near_zero_variance"] = bool(pd.notna(row["variance"]) and
    ↪row["variance"] <= LOW_VARIANCE_THRESHOLD)
    else:
        row["variance"] = np.nan
        row["zero_or_near_zero_variance"] = False
    low_variance_rows.append(row)

low_variance_table = pd.DataFrame(low_variance_rows).sort_values(
    ["is_constant", "zero_or_near_zero_variance", "near_constant_by_dominance",
    ↪"dominant_percent"],
    ascending=[False, False, False, False],
)
```

```
display(low_variance_table.head(35))

low_variance_candidates = low_variance_table[
    low_variance_table["is_constant"] |
    low_variance_table["zero_or_near_zero_variance"] |
    low_variance_table["near_constant_by_dominance"]
]
display(low_variance_candidates)
```

	column	dtype	unique_values	is_constant	dominant_value	\
31	Bwd PSH Flags	int64	1	True	0	
32	Fwd URG Flags	int64	1	True	0	
33	Bwd URG Flags	int64	1	True	0	
49	CWE Flag Count	int64	1	True	0	
56	Fwd Avg Bytes/Bulk	int64	1	True	0	
57	Fwd Avg Packets/Bulk	int64	1	True	0	
58	Fwd Avg Bulk Rate	int64	1	True	0	
59	Bwd Avg Bytes/Bulk	int64	1	True	0	
60	Bwd Avg Packets/Bulk	int64	1	True	0	
61	Bwd Avg Bulk Rate	int64	1	True	0	
45	RST Flag Count	int64	2	False	0	
50	ECE Flag Count	int64	2	False	0	
43	FIN Flag Count	int64	2	False	0	
71	Active Std	float64	5669	False	0.0	
30	Fwd PSH Flags	int64	2	False	0	
44	SYN Flag Count	int64	2	False	0	
75	Idle Std	float64	5857	False	0.0	
69	min_seg_size_forward	int64	8	False	20	
48	URG Flag Count	int64	2	False	0	
11	Bwd Packet Length Min	int64	343	False	0	
46	PSH Flag Count	int64	2	False	0	
70	Active Mean	float64	40409	False	0.0	
72	Active Max	int64	40313	False	0	
73	Active Min	int64	38393	False	0	
74	Idle Mean	float64	35285	False	0.0	
76	Idle Max	int64	33002	False	0	
77	Idle Min	int64	48018	False	0	
0	Destination Port	int64	23950	False	80	
38	Min Packet Length	int64	109	False	0	
7	Fwd Packet Length Min	int64	151	False	0	
47	ACK Flag Count	int64	2	False	1	
13	Bwd Packet Length Std	float64	9650	False	0.0	
9	Fwd Packet Length Std	float64	9555	False	0.0	
27	Bwd IAT Std	float64	113859	False	0.0	
51	Down/Up Ratio	int64	8	False	0	
	dominant_percent	near_constant_by_dominance		variance	\	
31	100.0000	True		0.0000		

32	100.0000	True	0.0000
33	100.0000	True	0.0000
49	100.0000	True	0.0000
56	100.0000	True	0.0000
57	100.0000	True	0.0000
58	100.0000	True	0.0000
59	100.0000	True	0.0000
60	100.0000	True	0.0000
61	100.0000	True	0.0000
45	99.9880	True	0.0001
50	99.9880	True	0.0001
43	99.7330	True	0.0027
71	97.4810	True	44,215,016,300.8259
30	96.6780	True	0.0321
44	96.6780	True	0.0321
75	90.0160	False	162,738,330,705,336.4062
69	88.1920	False	17.3622
48	85.9250	False	0.1209
11	74.0670	False	2,548.2877
46	64.8840	False	0.2278
70	64.4710	False	636,684,372,346.8243
72	64.4710	False	810,423,031,803.6925
73	64.4710	False	615,064,093,428.8525
74	64.4450	False	477,554,839,285,702.5000
76	64.4450	False	724,754,436,283,987.2500
77	64.4450	False	393,272,307,102,779.8125
0	60.6660	False	390,246,093.8857
38	53.5390	False	248.6208
7	51.9420	False	26,674.7809
47	50.4460	False	0.2500
13	49.4290	False	3,003,986.6307
9	48.6940	False	635,864.4201
27	45.1740	False	29,984,141,125,230.9062
51	43.1060	False	2.0471

	zero_or_near_zero_variance
31	True
32	True
33	True
49	True
56	True
57	True
58	True
59	True
60	True
61	True
45	False
50	False

43	False
71	False
30	False
44	False
75	False
69	False
48	False
11	False
46	False
70	False
72	False
73	False
74	False
76	False
77	False
0	False
38	False
7	False
47	False
13	False
9	False
27	False
51	False

	column	dtype	unique_values	is_constant	dominant_value \
31	Bwd PSH Flags	int64	1	True	0
32	Fwd URG Flags	int64	1	True	0
33	Bwd URG Flags	int64	1	True	0
49	CWE Flag Count	int64	1	True	0
56	Fwd Avg Bytes/Bulk	int64	1	True	0
57	Fwd Avg Packets/Bulk	int64	1	True	0
58	Fwd Avg Bulk Rate	int64	1	True	0
59	Bwd Avg Bytes/Bulk	int64	1	True	0
60	Bwd Avg Packets/Bulk	int64	1	True	0
61	Bwd Avg Bulk Rate	int64	1	True	0
45	RST Flag Count	int64	2	False	0
50	ECE Flag Count	int64	2	False	0
43	FIN Flag Count	int64	2	False	0
71	Active Std	float64	5669	False	0.0
30	Fwd PSH Flags	int64	2	False	0
44	SYN Flag Count	int64	2	False	0

	dominant_percent	near_constant_by_dominance	variance \
31	100.0000	True	0.0000
32	100.0000	True	0.0000
33	100.0000	True	0.0000
49	100.0000	True	0.0000
56	100.0000	True	0.0000

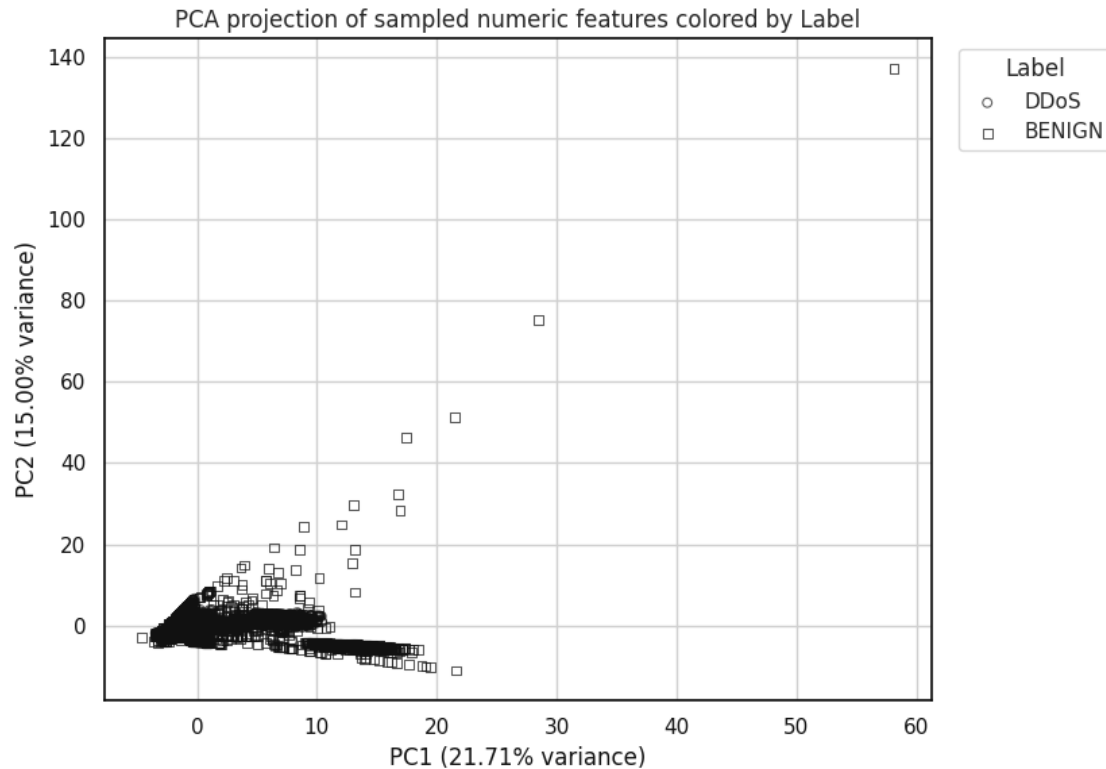
57	100.0000	True	0.0000
58	100.0000	True	0.0000
59	100.0000	True	0.0000
60	100.0000	True	0.0000
61	100.0000	True	0.0000
45	99.9880	True	0.0001
50	99.9880	True	0.0001
43	99.7330	True	0.0027
71	97.4810	True	44,215,016,300.8259
30	96.6780	True	0.0321
44	96.6780	True	0.0321

	zero_or_near_zero_variance
31	True
32	True
33	True
49	True
56	True
57	True
58	True
59	True
60	True
61	True
45	False
50	False
43	False
71	False
30	False
44	False

0.17 PCA 2D Visualization Colored By Label

```
[19]: pca_label_result = pca_projection(
    df,
    numeric_cols,
    TARGET_COL,
    ADVANCED_SAMPLE_SIZE,
    "PCA projection of sampled numeric features colored by Label",
)
```

	component	explained_variance_percent
0	PC1	21.7110
1	PC2	15.0000



0.18 Feature Ranking Using Mutual Information

```
[20]: feature_cols = [c for c in df.columns if c != TARGET_COL]
mi_data = sampled(df[[TARGET_COL] + feature_cols], FEATURE_RANK_SAMPLE_SIZE)
y_label = pd.Series(pd.factorize(mi_data[TARGET_COL].astype("string").
    ↪ fillna("<NA>"))[0], index=mi_data.index)

X_parts = []
discrete_features = []
encoded_feature_names = []

for col in feature_cols:
    series = mi_data[col]
    if pd.api.types.is_numeric_dtype(series):
        values = pd.to_numeric(series, errors="coerce").replace([np.inf, -np.
    ↪ inf], np.nan)
        fill_value = values.median() if values.notna().any() else 0
        X_parts.append(values.fillna(fill_value).to_numpy())
        discrete_features.append(False)
    else:
        encoded = pd.factorize(series.astype("string").fillna("<NA>"))[0]
        X_parts.append(encoded)
```

```

        discrete_features.append(True)
        encoded_feature_names.append(col)

if not X_parts:
    mi_feature_ranking = pd.DataFrame()
    display(Markdown("No predictors available for mutual information ranking."))
else:
    X_mi = np.column_stack(X_parts)
    mi_scores = mutual_info_classif(
        X_mi,
        y_label,
        discrete_features=np.array(discrete_features, dtype=bool),
        random_state=RANDOM_STATE,
    )
    mi_feature_ranking = (
        pd.DataFrame({
            "feature": encoded_feature_names,
            "mutual_information_with_label": mi_scores,
            "feature_type": ["categorical" if flag else "numeric" for flag in
↪discrete_features],
        })
        .sort_values("mutual_information_with_label", ascending=False)
        .reset_index(drop=True)
    )
    display(Markdown(f"Mutual information was computed on {len(mi_data):,}
↪sampled rows for speed and stability."))
    display(mi_feature_ranking.head(30))

plt.figure(figsize=(9, 6))
ax = sns.barplot(
    data=mi_feature_ranking.head(20),
    x="mutual_information_with_label",
    y="feature",
    hue="feature_type",
    dodge=False,
    palette=print_palette(2),
    edgecolor="#111111",
)
apply_bar_hatches(ax)
plt.title("Top mutual-information features against Label")
plt.xlabel("Mutual information")
plt.ylabel("")
plt.legend(title="Feature type")
plt.tight_layout()
plt.show()

```

Mutual information was computed on 60,000 sampled rows for speed and stability.

	feature	mutual_information_with_label	feature_type
0	Subflow Fwd Bytes	0.6512	numeric
1	Total Length of Fwd Packets	0.6511	numeric
2	Average Packet Size	0.5626	numeric
3	Total Length of Bwd Packets	0.5417	numeric
4	Subflow Bwd Bytes	0.5412	numeric
5	Bwd Packet Length Mean	0.5407	numeric
6	Fwd Header Length.1	0.5407	numeric
7	Avg Bwd Segment Size	0.5406	numeric
8	Fwd Header Length	0.5405	numeric
9	Destination Port	0.5367	numeric
10	Bwd Packet Length Max	0.5257	numeric
11	Init_Win_bytes_forward	0.4935	numeric
12	Avg Fwd Segment Size	0.4893	numeric
13	Fwd Packet Length Mean	0.4879	numeric
14	Fwd Packet Length Max	0.4845	numeric
15	Bwd Header Length	0.4720	numeric
16	Fwd IAT Max	0.4444	numeric
17	Fwd IAT Total	0.4411	numeric
18	Fwd IAT Mean	0.4372	numeric
19	Subflow Fwd Packets	0.4080	numeric
20	Total Fwd Packets	0.4061	numeric
21	Init_Win_bytes_backward	0.4048	numeric
22	Fwd IAT Std	0.3995	numeric
23	Packet Length Mean	0.3958	numeric
24	act_data_pkt_fwd	0.3953	numeric
25	Packet Length Std	0.3679	numeric
26	Packet Length Variance	0.3663	numeric
27	Bwd IAT Max	0.3544	numeric
28	Bwd IAT Total	0.3511	numeric
29	Bwd Packet Length Std	0.3450	numeric

/tmp/ipykernel_112443/1140206918.py:46: UserWarning: The palette list has more values (2) than needed (1), which may not be intended.

```
ax = sns.barplot(
```

